

Machine Learning – III (Reinforcement Learning)

Complete Study Material – Deep Research Enhanced Edition

Generated from course material with all numericals solved
Enhanced with additional diagrams, theoretical analysis, and worked examples

May 11, 2026

Contents

1	Module 1: Introduction to Reinforcement Learning	3
1.1	What is Reinforcement Learning?	3
1.2	RL vs Supervised vs Unsupervised Learning	3
1.3	Elements of Reinforcement Learning	3
1.3.1	Policy	4
1.3.2	Reward Signal	4
1.3.3	Value Function	4
1.3.4	Model-Based vs Model-Free	4
1.4	Approaches to Implementing RL	4
1.4.1	Value-Based	4
1.4.2	Policy-Based	5
1.4.3	Model-Based	5
1.5	The Exploration-Exploitation Dilemma	5
1.6	Evolutionary Methods vs RL	5
1.7	Immediate Reinforcement Learning	6
2	Module 2: Bandit Problems and Policy Gradient Approaches	6
2.1	The Multi-Armed Bandit Problem	6
2.1.1	Value-Action Based Method (Sample Average)	6
2.1.2	Greedy Method	8
2.1.3	Epsilon-Greedy Method	8
2.1.4	Upper Confidence Bound (UCB)	10
2.1.5	Thompson Sampling	11
2.2	Linear Reward-Penalty (LR-P) Algorithm	13
2.2.1	Action Probabilities and Update Rules	13
2.3	Policy Gradient Methods	14
2.3.1	Objective Function	15
2.3.2	The Policy Gradient Theorem	15
2.3.3	REINFORCE Algorithm	15
2.4	Comparison of Algorithms	17
3	Module 3: Full Reinforcement Learning and MDP	18
3.1	What is Full Reinforcement Learning?	18
3.2	Full RL vs Immediate RL	18
3.3	Agent-Environment Interface	19
3.4	Goals and Rewards	19
3.5	Returns and Discounting	19

3.5.1	Unified Notation	19
3.6	The Markov Property	20
3.7	Episodic vs Continuing Tasks	20
3.8	State Value Function $V_\pi(s)$	20
3.9	Action Value Function $Q_\pi(s, a)$	20
3.10	Relationship Between V and Q	20
3.11	Bellman Equations	22
3.11.1	Bellman Equation for V_π	22
3.11.2	Bellman Equation for Q_π	22
3.11.3	Bellman Optimality Equations	22
4	Module 4: Dynamic Programming and Monte Carlo Methods	24
4.1	Dynamic Programming in RL	24
4.2	Policy Evaluation (Prediction)	24
4.2.1	Visual Gridworld Reference	24
4.3	Policy Improvement	28
4.4	Policy Iteration	28
4.5	Value Iteration	29
4.6	Bootstrapping	29
4.7	Monte Carlo Methods	29
4.7.1	Types of Monte Carlo Prediction	29
4.8	On-Policy vs Off-Policy Learning	31
5	Module 5: Temporal Difference Learning	31
5.1	What is Temporal Difference Learning?	31
5.2	On-Policy vs Off-Policy TD Control	33
5.3	SARSA: On-Policy TD Control	34
5.4	Q-Learning: Off-Policy TD Control	35
6	Module 6: Function Approximation and Deep Reinforcement Learning	37
6.1	Drawbacks of Tabular Methods	37
6.2	Value Function Approximation	37
6.3	Gradient Descent Methods	37
6.4	Deep Q-Network (DQN)	40
6.5	Double DQN	40
6.6	Actor-Critic Architecture	40
6.7	A2C and A3C	41
6.8	DDPG and PPO	41
6.9	Algorithm Comparison Summary	43
6.10	Prioritized Experience Replay	43

1 Module 1: Introduction to Reinforcement Learning

1.1 What is Reinforcement Learning?

Reinforcement Learning (RL) is a paradigm of machine learning where an **agent** learns to make decisions by interacting with an **environment**. Unlike supervised learning, where the agent learns from labeled input-output pairs, or unsupervised learning, where the agent discovers hidden patterns in unlabeled data, reinforcement learning is driven by a **reward signal**. The agent takes actions in the environment, receives feedback in the form of rewards or penalties, and over time learns a strategy (called a **policy**) that maximizes the cumulative reward.

The fundamental idea behind RL is learning from interaction with the environment, which mirrors how humans and animals naturally learn. Consider a child encountering a fireplace for the first time: the child feels warmth at a safe distance (positive reward), but upon touching the fire, experiences pain (negative reward). Through this interaction, the child learns to stay at an appropriate distance from fire. Reinforcement Learning is the computational formulation of this natural learning process.

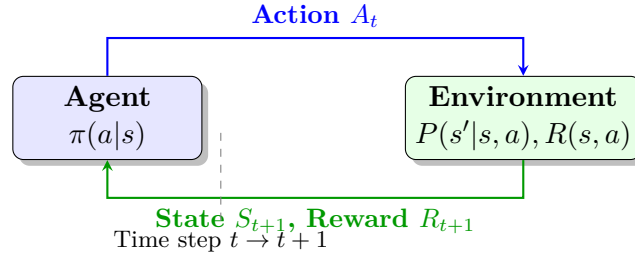
Key Insight: The RL Loop

The Reinforcement Learning process is modeled as a loop:

1. The **Agent** receives a **state** S_t from the **Environment**.
2. Based on S_t , the agent selects an **action** A_t .
3. The environment transitions to a new **state** S_{t+1} and provides a **reward** R_{t+1} .
4. The agent uses this reward to update its policy and repeats.

1.1.1 Detailed Agent-Environment Interaction Diagram

The following diagram illustrates the complete interaction cycle with mathematical notation:



Mathematical Formulation of the RL Loop

At each time step t :

1. **Observation:** Agent observes state $S_t \in \mathcal{S}$
2. **Decision:** Agent samples action $A_t \sim \pi(\cdot|S_t)$ where $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ is the policy
3. **Transition:** Environment samples next state $S_{t+1} \sim P(\cdot|S_t, A_t)$
4. **Reward:** Environment generates reward $R_{t+1} = R(S_t, A_t, S_{t+1})$
5. **Learning:** Agent updates policy π or value estimates based on $(S_t, A_t, R_{t+1}, S_{t+1})$

Trajectory (Episode): $\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T)$ where T is the terminal time step.

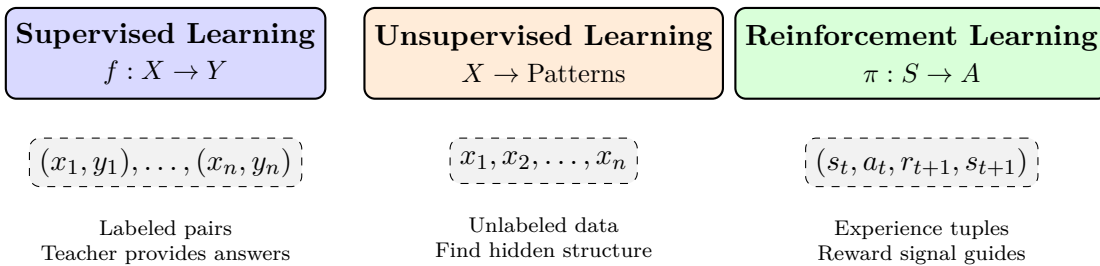
1.2 RL vs Supervised vs Unsupervised Learning

Supervised learning involves learning from labeled data with a clear input-output mapping. The training data provides explicit correct answers, and the model learns to map inputs to these pre-determined outputs. This is suitable for classification and regression tasks where ground truth is available.

Unsupervised learning deals with discovering patterns or structures in unlabeled data without explicit output labels. The agent explores the data to find inherent groupings, clusters, or distributions. This is useful for tasks like clustering, dimensionality reduction, and anomaly detection.

Reinforcement learning focuses on training agents to make sequential decisions in an environment, optimizing for a cumulative reward signal through interaction and feedback. The agent does not receive correct actions; instead, it must discover which actions lead to the highest long-term reward through trial and error. This makes RL particularly suitable for control tasks, game playing, robotics, and sequential decision-making problems.

1.2.1 Comprehensive Learning Paradigm Comparison



Worked Example: Identifying the Learning Paradigm

For each scenario, identify which learning paradigm applies and justify your answer.

Scenario 1: Training a model to classify emails as spam or ham using a dataset of 10,000 labeled emails.

- **Answer: Supervised Learning** — The dataset contains input-output pairs (emails with spam/ham labels), and the goal is to learn a mapping from email content to classification.

Scenario 2: A robot learning to balance a pole by receiving +1 reward for each timestep the pole stays upright and -10 when it falls.

- **Answer: Reinforcement Learning** — The robot learns through trial and error with reward feedback, without labeled correct actions. The goal is to maximize cumulative reward over time.

Scenario 3: Grouping customers into segments based on purchasing patterns without pre-defined categories.

- **Answer: Unsupervised Learning** — No labels exist; the algorithm discovers natural groupings in the data.

Scenario 4: An agent learning to play chess by playing thousands of games against itself, receiving +1 for wins, -1 for losses, and 0 for draws.

- **Answer: Reinforcement Learning** — Sequential decision-making with delayed rewards (win/loss comes after many moves).

1.3 Elements of Reinforcement Learning

A reinforcement learning system has four main subelements:

- **Policy:** A mapping from perceived states to actions. It defines the agent's behavior.
- **Reward Signal:** A scalar feedback signal indicating how well the agent is doing at each step.
- **Value Function:** A prediction of future rewards, indicating the long-term desirability of states.
- **Model of the Environment** (optional): A representation that mimics the behavior of the environment, used for planning.

1.3.1 Policy

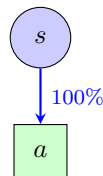
A **policy** can be defined as a way an agent behaves at a given time. It maps the perceived states of the environment to the actions taken in those states. A policy is the core element of RL as it alone can define the behavior of the agent. In some cases, it may be a simple function or a lookup table; for other cases, it may involve general computation as a search process.

Policies can be either:

- **Deterministic policy:** $a = \pi(s)$ — the policy maps each state to exactly one action. This is easy to interpret and implement, and is suitable for tasks where the same action should always be taken for the same state. For example, in chess, the best move for a given board configuration is always the same.
- **Stochastic policy:** $\pi(a | s) = P[A_t = a | S_t = s]$ — the policy returns a probability distribution over actions. This can capture uncertainty in the environment and is useful for exploration. For example, in Rock-Paper-Scissors, the best strategy is stochastic, choosing each action a third of the time.

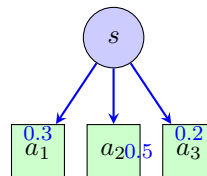
Deterministic Policy

$$\pi(s) = a$$



Stochastic Policy

$$\pi(a|s)$$



Worked Example: Computing Action Probabilities

Given: A stochastic policy defined by the following probability table:

State	$P(a_1 s)$	$P(a_2 s)$	$P(a_3 s)$
s_1	0.6	0.3	0.1
s_2	0.2	0.5	0.3
s_3	0.0	0.0	1.0

Questions and Solutions:

1. What is the probability of taking action a_2 in state s_1 ?

$$\pi(a_2|s_1) = 0.3$$

2. In which state is the policy deterministic?

State s_3 is deterministic because $\pi(a_3|s_3) = 1.0$ (probability 1 for a single action)

3. Compute the entropy of the policy in state s_1 (measure of randomness).

$$H(\pi(\cdot|s_1)) = - \sum_a \pi(a|s_1) \log_2 \pi(a|s_1) = -(0.6 \log_2 0.6 + 0.3 \log_2 0.3 + 0.1 \log_2 0.1)$$

$$= -(0.6(-0.737) + 0.3(-1.737) + 0.1(-3.322)) = 0.442 + 0.521 + 0.332 = \mathbf{1.295} \text{ bits}$$

Higher entropy = more exploration; lower entropy = more exploitation.

1.3.2 Reward Signal

A **reward** is a scalar feedback signal indicating how well the agent is doing at step t . The agent's sole objective is to maximize the total reward it receives over the long run. The reward signal is the primary basis for altering the policy.

Different forms of reward functions:

- $R(s)$: Reward for simply being in state s .
- $R(s, a)$: Reward for being in state s and taking action a .
- $R(s, a, s')$: Reward for being in state s , taking action a , and transitioning to state s' .

1.3.3 Value Function

Value functions estimate how good it is for an agent to be in a given state, or how good it is to perform a given action in a given state. This notion of “how good” is given in terms of **expected return** — the total discounted reward the agent expects to accumulate from that point onward.

Value functions are defined with respect to **policies**: since the way an agent acts is influenced by the policy it is following, the value of a state depends on which policy the agent is using.

1.3.4 Model-Based vs Model-Free

The last element is the **model**, which mimics the behavior of the environment. With the model, one can make inferences about how the environment will behave — predicting the next state and reward given a state and action.

- **Model-Based RL**: The agent learns or is given a model of the environment and uses it for planning. The agent can simulate future situations before experiencing them.

- **Model-Free RL:** The agent learns purely from experience without building an explicit model of the environment dynamics.

1.4 Approaches to Implementing RL

There are three main approaches to implementing reinforcement learning:

1.4.1 Value-Based

The value-based approach aims to find the **optimal value function**, which is the maximum value achievable at a state under any policy. The agent expects the long-term return at any state under the optimal policy π^* . The agent will follow a policy that selects actions leading to states with higher values.

Key algorithms: Q-Learning, SARSA, DQN.

1.4.2 Policy-Based

The policy-based approach directly searches for the optimal policy that maximizes future rewards, without explicitly using a value function. The agent tries to apply a policy such that the action performed in each step helps maximize future reward. The policy can be deterministic or stochastic.

Key algorithms: REINFORCE, Actor-Critic, PPO.

1.4.3 Model-Based

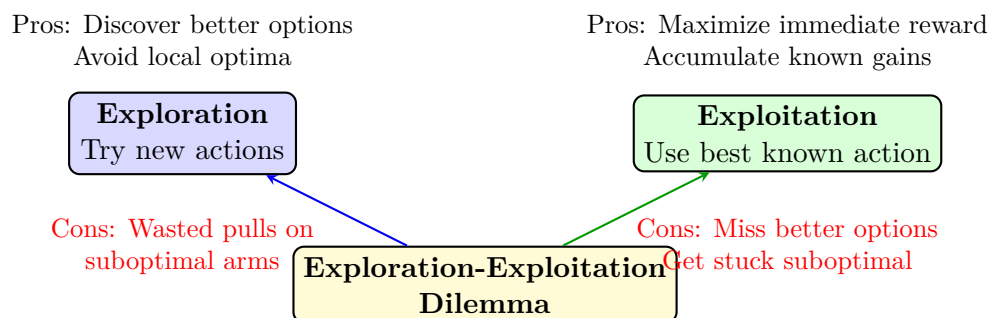
The model-based approach creates a virtual model of the environment. The agent explores this model to learn optimal behavior. This provides benefits: the agent can interact with the environment a few times, then the model can simulate subsequent iterations without needing further interaction.

1.5 The Exploration-Exploitation Dilemma

A fundamental challenge in reinforcement learning is the **exploration-exploitation dilemma**:

- **Exploitation** is the greedy approach where agents try to get more rewards by using estimated values. The agent makes the best decision based on current information. This is safe but may miss better options.
- **Exploration** is the approach where agents focus on improving their knowledge about each action instead of getting more immediate rewards, so they can achieve long-term benefits. The agent gathers more information to make the best overall decision. This is risky but can discover better strategies.

The dilemma arises because the agent must balance between exploiting what it already knows and exploring new actions that might yield higher rewards. Too much exploitation leads to suboptimal policies; too much exploration wastes time on poor actions.



Example: Consider a multi-armed bandit problem where an agent must choose between multiple slot machines (arms). Each arm has an unknown probability of paying out. The agent must decide whether to keep pulling the arm with the highest observed payout (exploitation) or try other arms to see if they are better (exploration).

Worked Example: Regret Analysis in Exploration-Exploitation

Given: A 3-armed bandit with true mean rewards: $q^*(1) = 0.3$, $q^*(2) = 0.5$, $q^*(3) = 0.7$.

Scenario A - Pure Exploitation (Greedy): The agent happens to try arm 2 first, gets lucky with a reward of 1. Believing arm 2 is best, it keeps pulling arm 2 forever.

Scenario B - Balanced Exploration: The agent explores all arms sufficiently and discovers that arm 3 is actually the best.

Calculate: The cumulative regret after 1000 pulls for each scenario.

Solution:

Optimal expected reward per pull: $q^*(3) = 0.7$

Scenario A (stuck on arm 2):

- Expected reward per pull: $q^*(2) = 0.5$
- Regret per pull: $0.7 - 0.5 = 0.2$
- **Cumulative regret after 1000 pulls:** $1000 \times 0.2 = 200$

Scenario B (finds optimal arm 3):

- First 100 pulls: exploring (assume uniform exploration, average reward = 0.5)
- Next 900 pulls: exploiting arm 3 (optimal, reward = 0.7)
- Exploration regret: $100 \times (0.7 - 0.5) = 20$
- Exploitation regret: $900 \times (0.7 - 0.7) = 0$
- **Total regret:** $20 + 0 = 20$

Interpretation: The greedy approach that got “stuck” on a suboptimal arm suffers 10x more regret (200 vs 20). This demonstrates why some exploration is essential, even though it has short-term costs.

Worked Example: Calculating the Exploration Gap

Given: An agent has estimates $\hat{Q}(a)$ for three actions after limited exploration:

- $\hat{Q}(A) = 0.8$ (explored 10 times)
- $\hat{Q}(B) = 0.7$ (explored 100 times, very confident)
- $\hat{Q}(C) = 0.6$ (explored 5 times, high uncertainty)

Question: Should the agent exploit (choose A) or explore C further?

Analysis:

1. **Exploitation choice:** Action A gives expected reward 0.8
2. **Exploration value:** Action C has high uncertainty — if its true value is higher than 0.8, exploring now could lead to better long-term rewards
3. **Information value:** The uncertainty in C's estimate means there's a non-trivial probability that $Q^*(C) > 0.8$

Decision Framework: The value of exploration can be approximated by:

$$\text{Value of exploring C} = P(Q^*(C) > Q^*(A)) \times (Q^*(C) - Q^*(A))$$

Even if $P(Q^*(C) > 0.8) = 0.2$ and $Q^*(C) = 0.9$ if true, the expected value of information is $0.2 \times (0.9 - 0.8) = 0.02$.

Conclusion: In this case, exploitation (Action A) is likely optimal given the large gap and C's low initial estimate. However, if $\hat{Q}(C)$ were 0.75 instead of 0.6, exploration would be more strongly justified.

1.6 Evolutionary Methods vs RL

Evolutionary methods function through a selection process where the least fit members of a population are eliminated, while the fit members survive and continue until better solutions are determined. Evolutionary algorithms mimic biological processes (selection, mutation, crossover) to solve complex problems.

Reinforcement Learning	Evolutionary Algorithms
Single agent learns from environment	Population of solutions evolves
Learns from both positive and negative feedback	Only best solutions survive; negative info discarded
Better for adapting to new environments	Better for pure optimization
Uses value functions and policies	Uses fitness scores and genetic operators

Example: Teaching a robot to walk:

- **RL approach:** The robot takes random steps, receives positive rewards for good actions (moving forward), negative rewards for bad actions (falling). Over time, it learns from both successes and failures.
- **EA approach:** Start with 100 robots, each with different walking strategies. Test performance, eliminate worst-performing robots. Best robots “reproduce” (combine strategies) to create a new generation. Repeat for multiple generations.

1.7 Immediate Reinforcement Learning

Immediate RL is a type of reinforcement learning where the agent receives immediate rewards or penalties for its actions at each time step without considering long-term consequences. The agent's

goal is to maximize its cumulative rewards over time, but each decision is evaluated purely on its immediate outcome.

This contrasts with **full reinforcement learning**, where the agent must consider delayed rewards and the long-term consequences of its actions. Immediate RL is simpler but less powerful for tasks requiring planning and foresight.

Example: In the game Subway Surfer, the agent receives a positive reward when it collects a coin and a negative reward when it runs into an obstacle. Immediate RL focuses on learning effective actions based on these immediate rewards without considering the entire game’s strategy and planning ahead.

2 Module 2: Bandit Problems and Policy Gradient Approaches

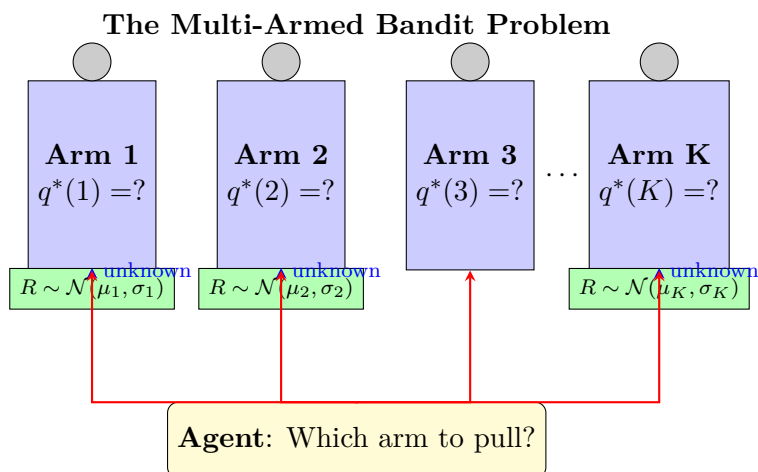
2.1 The Multi-Armed Bandit Problem

The **Multi-Armed Bandit (MAB) problem** serves as a foundational concept in reinforcement learning, illustrating the challenges and strategies of decision-making under uncertainty. The name comes from the one-armed bandit (slot machine): imagine a row of slot machines, each with a different probability of paying out. Your goal is to maximize your total reward over a fixed number of pulls.

Formally, a K -armed bandit problem has K actions (arms). Pulling arm a at time t yields a reward R_t drawn from a probability distribution with an unknown mean $q^*(a)$. The agent’s goal is to maximize the cumulative reward over T time steps.

The core challenge is the **exploration-exploitation trade-off**:

- **Exploitation:** Choose the arm with the highest estimated reward to maximize immediate gain.
- **Exploration:** Try other arms to gather information about their true reward distributions, which may lead to higher long-term rewards.



Worked Example: Computing True Values and Sample Estimates

Given: A 3-armed bandit where each arm's reward follows a normal distribution:

- Arm 1: $R \sim \mathcal{N}(5, 2)$ — mean $\mu_1 = 5$, variance $\sigma_1^2 = 2$
- Arm 2: $R \sim \mathcal{N}(7, 1)$ — mean $\mu_2 = 7$, variance $\sigma_2^2 = 1$
- Arm 3: $R \sim \mathcal{N}(4, 3)$ — mean $\mu_3 = 4$, variance $\sigma_3^2 = 3$

Questions:

1. What are the true action values?

$$Q^*(1) = 5, \quad Q^*(2) = 7, \quad Q^*(3) = 4$$

Arm 2 is the **optimal arm** with true value 7.

2. After 10 pulls, the agent observed the following sample averages:

$$\hat{Q}(1) = 5.3, \quad \hat{Q}(2) = 6.5, \quad \hat{Q}(3) = 5.8$$

Why does the greedy agent choose the wrong arm?

Analysis: The greedy agent would choose Arm 3 ($\hat{Q}(3) = 5.8$) instead of the true optimal Arm 2 ($\hat{Q}(2) = 6.5$) — wait, actually with these estimates, it would choose Arm 2. Let us correct: if the estimates were:

$$\hat{Q}(1) = 5.3, \quad \hat{Q}(2) = 5.5, \quad \hat{Q}(3) = 6.0$$

Then the greedy agent chooses Arm 3, even though Arm 2 is truly optimal. This is due to **sampling variance** — with only 10 samples, the estimates have high uncertainty.

3. Compute the standard error of the mean for each arm after n samples.

$$\text{SE}(\hat{Q}(a)) = \frac{\sigma_a}{\sqrt{n}}$$

For Arm 2 with 10 samples: $\text{SE} = \frac{1}{\sqrt{10}} \approx 0.316$

This means the estimate $\hat{Q}(2) = 6.5$ could easily be off by ± 0.6 (2 standard errors), explaining why the agent might mistakenly think another arm is better.

2.1.1 Value-Action Based Method (Sample Average)

We denote the **true value** of action a as $Q^*(a)$, and the **estimated value** at time step t as $Q_t(a)$. Recall that the true value of an action is the mean reward received when that action is selected.

The estimated action value using the **sample-average method** is:

$$Q_t(a) = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{I}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbb{I}_{A_i=a}}$$

where $\mathbb{I}_{A_i=a}$ is an indicator function that is 1 if action a was selected at step i , and 0 otherwise. This is simply the average of all rewards received when action a was taken.

Worked Example 1: Sample Average Action Value Estimation

Given: A 3-armed bandit with the following rewards observed over 5 time steps:

Step	Arm	Reward
1	1	4
2	2	5
3	1	6
4	3	3
5	2	7

Calculate: $Q_6(1)$, $Q_6(2)$, $Q_6(3)$ using sample average.

Solution:

1. Arm 1 selected at steps 1 and 3: rewards 4 and 6

$$Q_6(1) = \frac{4 + 6}{2} = \frac{10}{2} = 5.0$$

2. Arm 2 selected at steps 2 and 5: rewards 5 and 7

$$Q_6(2) = \frac{5 + 7}{2} = \frac{12}{2} = 6.0$$

3. Arm 3 selected at step 4: reward 3

$$Q_6(3) = \frac{3}{1} = 3.0$$

Answer: $Q_6(1) = 5.0$, $Q_6(2) = 6.0$, $Q_6(3) = 3.0$

Interpretation: Based on current estimates, Arm 2 appears best. However, Arm 3 has only been tried once, so its estimate has high uncertainty.

Worked Example 2: Incremental Update with Sample Average

Given: $Q_5(1) = 5.0$ (2 samples). At step 5, arm 1 is selected again with reward $R_5 = 8$.

Calculate: $Q_6(1)$ using incremental update.

Solution: The incremental update formula:

$$Q_{n+1} = Q_n + \frac{1}{n}(R_n - Q_n)$$

1. Current count for arm 1: $n = 2$
2. New reward: $R_5 = 8$
3. $Q_6(1) = 5.0 + \frac{1}{3}(8 - 5.0) = 5.0 + \frac{3}{3} = 5.0 + 1.0 = 6.0$

Answer: $Q_6(1) = 6.0$

Verification: Full sample average = $\frac{4+6+8}{3} = \frac{18}{3} = 6.0$. Matches.

Worked Example 3: Non-Stationary with Constant Step Size

Given: $Q_t(1) = 5.0$. Step size $\alpha = 0.2$. New reward $R_t = 3$.

Calculate: Updated $Q_{t+1}(1)$ using constant step size.

Solution:

1. The constant step-size update rule:

$$Q_{t+1}(a) = Q_t(a) + \alpha[R_t - Q_t(a)]$$

2. $Q_{t+1}(1) = 5.0 + 0.2(3 - 5.0) = 5.0 + 0.2(-2.0) = 5.0 - 0.4 = 4.6$

Answer: $Q_{t+1}(1) = 4.6$

Interpretation: With $\alpha = 0.2$, the estimate shifts only 20% of the way toward the new reward. This makes the estimate more stable but slower to adapt. For non-stationary environments (where true reward distributions change), a larger α would adapt faster.

2.1.2 Greedy Method

The simplest action selection rule is to select the action with the highest estimated action value:

$$A_t = \arg \max_a Q_t(a)$$

This greedy action selection always exploits current knowledge to maximize immediate reward; it spends no time sampling apparently inferior actions to see if they might really be better. The problem is clear: if the initial estimates are wrong, the agent may get stuck with a suboptimal choice forever.

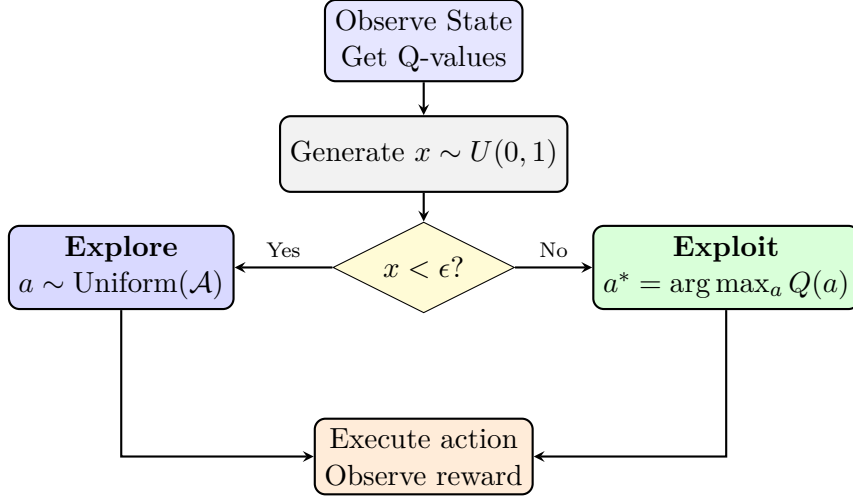
2.1.3 Epsilon-Greedy Method

The **ϵ -greedy method** is a simple yet effective way to balance exploration and exploitation. With probability $1 - \epsilon$, the agent selects the greedy action (exploitation). With probability ϵ , the agent selects a random action (exploration).

- $\epsilon = 0$: Pure exploitation (always choose the greedy action).
- $\epsilon = 1$: Pure exploration (always choose a random action).
- Common values: $\epsilon = 0.1$, 0.01 , or 0.001 , depending on the problem.

An important property: in the limit as the number of plays increases, every action will be sampled infinitely, guaranteeing that $N_t(a) \rightarrow \infty$ for all a , and thus ensuring that all $Q_t(a)$ converge to $q^*(a)$. The probability of selecting the optimal action converges to greater than $1 - \epsilon$.

ϵ -Greedy Decision Process



Theoretical Analysis of ϵ -Greedy Convergence

Convergence Guarantee: For a K -armed bandit with bounded rewards, ϵ -greedy satisfies:

1. **Exploration Guarantee:** Every arm a is pulled infinitely often as $t \rightarrow \infty$ because:

$$P(\text{select } a \text{ at time } t) \geq \frac{\epsilon}{K} > 0$$

2. **Consistency:** Sample averages converge to true values:

$$Q_t(a) \xrightarrow{a.s.} q^*(a) \quad \text{as } t \rightarrow \infty$$

3. **Asymptotic Suboptimality:** The policy never fully converges to optimal because it keeps exploring with probability ϵ even after identifying the best arm.

$$\lim_{t \rightarrow \infty} P(\text{select optimal arm}) = (1 - \epsilon) + \frac{\epsilon}{K} < 1$$

4. **Linear Regret:** The cumulative regret grows linearly with time:

$$R_T = \sum_{t=1}^T (q^*(a^*) - q^*(a_t)) = \Omega(\epsilon T)$$

Worked Example 4: Epsilon-Greedy Action Selection

Given: Three arms with estimated values $Q(1) = 5.2$, $Q(2) = 6.0$, $Q(3) = 4.8$. $\epsilon = 0.1$.

Calculate: Probability of selecting each arm.

Solution:

1. With probability $1 - \epsilon = 0.9$: select the greedy action (Arm 2, highest Q-value).
2. With probability $\epsilon = 0.1$: select uniformly at random from all 3 arms, so each arm gets $\frac{0.1}{3} = 0.0333$.

Total probabilities:

- $P(\text{Arm 1}) = 0.0333$
- $P(\text{Arm 2}) = 0.9 + 0.0333 = 0.9333$
- $P(\text{Arm 3}) = 0.0333$

Interpretation: The greedy arm gets selected 93% of the time, while each non-greedy arm gets 3.3% exploration probability.

Worked Example 5: Epsilon-Greedy with Simulation

Given: $\epsilon = 0.2$, 4 arms with $Q = [3, 5, 4, 2]$. Simulate 10 selections.

Random numbers drawn: 0.15, 0.85, 0.05, 0.92, 0.40, 0.78, 0.12, 0.65, 0.30, 0.88

Solution:

1. Random 0.15 < 0.2: **explore** → random arm (say arm 3)
2. Random 0.85 > 0.2: **exploit** → arm 2 (max Q = 5)
3. Random 0.05 < 0.2: **explore** → random arm (say arm 1)
4. Random 0.92 > 0.2: **exploit** → arm 2
5. Random 0.40 > 0.2: **exploit** → arm 2
6. Random 0.78 > 0.2: **exploit** → arm 2
7. Random 0.12 < 0.2: **explore** → random arm (say arm 4)
8. Random 0.65 > 0.2: **exploit** → arm 2
9. Random 0.30 > 0.2: **exploit** → arm 2
10. Random 0.88 > 0.2: **exploit** → arm 2

Result: Arm 2 selected 7 times, Arms 1, 3, 4 selected once each (exploration).

Worked Example 6: Comparing Epsilon Values

Given: True values $q^* = [5, 8, 6]$. After 3 initial pulls (one each arm), sample averages are $[7, 5, 6]$.

Calculate: Best action for $\epsilon = 0$ vs $\epsilon = 0.1$.

Solution:

1. With $\epsilon = 0$ (pure greedy): Always select Arm 1 ($Q = 7$), even though the true best is Arm 2 ($q^* = 8$). The agent is stuck on a lucky early pull.
2. With $\epsilon = 0.1$: 90% of the time selects Arm 1, but 10% explores randomly, giving a chance to discover Arm 2's true value.

Interpretation: Lower ϵ gives more exploitation but risks suboptimal convergence. Higher ϵ explores more but wastes pulls on poor arms.

2.1.4 Upper Confidence Bound (UCB)

The ϵ -greedy method explores randomly, which is wasteful. **Upper Confidence Bound (UCB)** is a smarter exploration strategy that selects actions based on both their estimated value and the uncertainty in that estimate. The formula is:

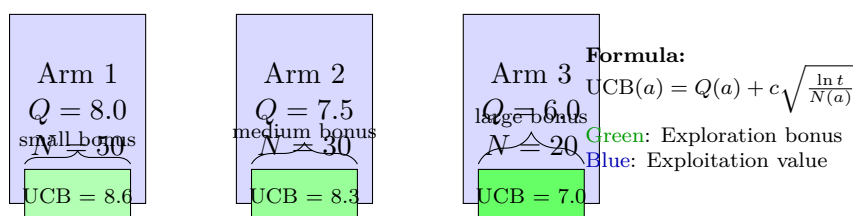
$$A_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

where:

- $Q_t(a)$: Current estimated value (exploitation term)
- c : Exploration constant (hyperparameter)
- t : Total number of time steps so far
- $N_t(a)$: Number of times action a has been selected

The square root term is a measure of uncertainty. It is proportional to t (more total steps means more information overall) and inversely proportional to $N_t(a)$ (actions tried less often have higher uncertainty). As time increases, the denominator dominates and the uncertainty term shrinks for well-explored actions.

UCB: Balancing Exploitation vs Exploration



UCB Theoretical Foundation

Hoeffding's Inequality: For bounded random variables $X_i \in [0, 1]$ with true mean μ and sample mean $\hat{\mu}_n$ after n samples:

$$P(\mu > \hat{\mu}_n + \epsilon) \leq e^{-2n\epsilon^2}$$

Setting $\delta = e^{-2n\epsilon^2}$ and solving for ϵ :

$$\epsilon = \sqrt{\frac{\ln(1/\delta)}{2n}}$$

UCB1 Algorithm: Setting $\delta = t^{-4}$ (confidence parameter that decreases with time):

$$\text{UCB}_t(a) = \hat{Q}_t(a) + \sqrt{\frac{2 \ln t}{N_t(a)}}$$

Regret Bound: UCB achieves logarithmic regret:

$$R_T \leq 8 \sum_{a: q^*(a) < q^*(a^*)} \frac{\ln T}{\Delta_a} + O(1)$$

where $\Delta_a = q^*(a^*) - q^*(a)$ is the suboptimality gap.

This is exponentially better than ϵ -greedy's linear regret $R_T = \Omega(T)$.

Worked Example: UCB Dynamics Over Time

Given: Two-armed bandit with $q^*(1) = 0.6$, $q^*(2) = 0.4$. True optimal: Arm 1. Track UCB scores over first 10 pulls with $c = \sqrt{2}$.

Solution - Step by Step:

Initialization ($t = 1$): Both arms untried. UCB uses initialization bonus.

$t = 2$: Try Arm 1, get reward 1 (lucky). $Q(1) = 1.0$, $N(1) = 1$.

$$\text{UCB}(1) = 1.0 + \sqrt{2 \ln 2 / 1} = 1.0 + 1.18 = 2.18$$

$$\text{UCB}(2) = 0 + \sqrt{2 \ln 2 / 0} = +\infty \text{ (unexplored)}$$

Select: Arm 2 (exploration priority).

$t = 3$: Try Arm 2, get reward 0. $Q(2) = 0$, $N(2) = 1$.

$$\text{UCB}(1) = 1.0 + \sqrt{2 \ln 3 / 1} = 1.0 + 1.48 = 2.48$$

$$\text{UCB}(2) = 0 + \sqrt{2 \ln 3 / 1} = 0 + 1.48 = 1.48$$

Select: Arm 1 (higher UCB).

$t = 4$: Try Arm 1, get reward 1. $Q(1) = 1.0$, $N(1) = 2$.

$$\text{UCB}(1) = 1.0 + \sqrt{2 \ln 4 / 2} = 1.0 + 0.83 = 1.83$$

$$\text{UCB}(2) = 0 + \sqrt{2 \ln 4 / 1} = 0 + 1.67 = 1.67$$

Select: Arm 1 (still higher, but closer).

Pattern: As t grows, the exploration bonus shrinks as $O(1/\sqrt{N(a)})$, and the algorithm increasingly favors the arm with higher estimated value.

Worked Example 7: UCB Score Calculation

Given: Three arms at $t = 100$. Values: $Q(1) = 8.0$, $N(1) = 50$; $Q(2) = 7.5$, $N(2) = 30$; $Q(3) = 6.0$, $N(3) = 20$. $c = 2$.

Calculate: UCB scores for each arm and determine the best action.

Solution:

1. Arm 1: $8.0 + 2\sqrt{\frac{\ln 100}{50}} = 8.0 + 2\sqrt{\frac{4.605}{50}} = 8.0 + 2\sqrt{0.0921} = 8.0 + 2(0.303) = 8.0 + 0.606 = 8.606$
2. Arm 2: $7.5 + 2\sqrt{\frac{\ln 100}{30}} = 7.5 + 2\sqrt{\frac{4.605}{30}} = 7.5 + 2\sqrt{0.1535} = 7.5 + 2(0.392) = 7.5 + 0.784 = 8.284$
3. Arm 3: $6.0 + 2\sqrt{\frac{\ln 100}{20}} = 6.0 + 2\sqrt{\frac{4.605}{20}} = 6.0 + 2\sqrt{0.2303} = 6.0 + 2(0.480) = 6.0 + 0.960 = 6.960$

Answer: UCB scores: Arm 1 = 8.606, Arm 2 = 8.284, Arm 3 = 6.960. Best action: **Arm 1** (highest UCB score).

Interpretation: Although Arm 2 has higher estimated uncertainty (lower N), Arm 1's higher Q -value compensates, giving it the highest UCB score.

Worked Example 8: UCB Early-Stage Behavior

Given: Four arms, $t = 4$, all $Q(a) = 0$ (initial estimates), $N(a) = 1$ for all arms (each tried once). $c = 2$.

Calculate: UCB scores.

Solution:

$$\text{UCB}(a) = 0 + 2\sqrt{\frac{\ln 4}{1}} = 2\sqrt{1.386} = 2(1.178) = 2.356$$

Answer: All arms have $\text{UCB} = 2.356$.

Interpretation: In early stages with equal estimates, UCB breaks ties by random selection (or lowest index). After more pulls, the arm with highest true value will accumulate higher $Q(a)$ and be selected more. UCB automatically reduces exploration as confidence grows.

Worked Example 9: UCB vs Epsilon-Greedy Comparison

Given: 5 arms at $t = 1000$. True values: $[8, 6, 7, 5, 9]$. Estimates after 200 pulls each: $Q = [7.8, 6.1, 7.0, 5.2, 8.5]$. Counts: $N = [200, 200, 200, 200, 200]$.

Calculate: Next action for ϵ -greedy ($\epsilon = 0.1$) vs UCB ($c = 2$).

Solution:

1. ϵ -greedy: Greedy arm is arm 5 ($Q = 8.5$). With 90% probability selects arm 5; with 10% selects random arm (arms 1-4, each 2.5%).
2. UCB: $\ln(1000) = 6.908$
 - Arm 1: $7.8 + 2\sqrt{6.908/200} = 7.8 + 0.372 = 8.172$
 - Arm 2: $6.1 + 2\sqrt{6.908/200} = 6.1 + 0.372 = 6.472$
 - Arm 3: $7.0 + 2\sqrt{6.908/200} = 7.0 + 0.372 = 7.372$
 - Arm 4: $5.2 + 2\sqrt{6.908/200} = 5.2 + 0.372 = 5.572$
 - Arm 5: $8.5 + 2\sqrt{6.908/200} = 8.5 + 0.372 = 8.872$

Answer: Both methods select arm 5. But UCB's exploration bonus means arms with close values (arm 1: 8.172 vs arm 5: 8.872) get more systematic exploration than random epsilon-greedy.

2.1.5 Thompson Sampling

Thompson Sampling (also called Bayesian Bandits) takes a probabilistic Bayesian approach to the bandit problem. Instead of maintaining point estimates $Q_t(a)$, it maintains a **probability distribution** over each arm's expected reward. The algorithm:

1. Maintain a $\text{Beta}(\alpha_a, \beta_a)$ distribution for each arm a .
2. Sample a random value from each distribution.
3. Select the arm with the highest sampled value.

For binary rewards (Bernoulli bandits), the Beta distribution is ideal. The parameters update as:

- If reward = 1: $\alpha_a \leftarrow \alpha_a + 1$
- If reward = 0: $\beta_a \leftarrow \beta_a + 1$

Initially, with no prior knowledge, we set $\alpha_a = 1$, $\beta_a = 1$ (uniform prior). As more data is collected, the distribution becomes narrower and more accurate.

Worked Example 10: Thompson Sampling with Beta Updates

Given: Two arms. Initial priors: Arm 1: Beta(1,1), Arm 2: Beta(1,1).
Observations:

- Arm 1: 3 successes (reward=1), 2 failures (reward=0)
- Arm 2: 1 success, 4 failures

Calculate: Posterior distributions and mean estimates.

Solution:

1. Arm 1: $\alpha_1 = 1 + 3 = 4$, $\beta_1 = 1 + 2 = 3$

$$\text{Mean} = \frac{\alpha_1}{\alpha_1 + \beta_1} = \frac{4}{7} = 0.571$$

2. Arm 2: $\alpha_2 = 1 + 1 = 2$, $\beta_2 = 1 + 4 = 5$

$$\text{Mean} = \frac{2}{7} = 0.286$$

Answer: Arm 1: Beta(4,3), mean = 0.571. Arm 2: Beta(2,5), mean = 0.286.

Interpretation: Thompson Sampling would sample from these distributions. Arm 1 has higher mean but wider distribution. There is still some probability Arm 2's sample could exceed Arm 1's, allowing exploration.

Worked Example 11: Thompson Sampling Decision

Given: Arm 1: Beta(4,3). Arm 2: Beta(2,5). Sampled values: Arm 1 = 0.62, Arm 2 = 0.35.

Question: Which arm is selected? What if next sample gives Arm 1 = 0.45, Arm 2 = 0.51?

Solution:

1. First sample: Arm 1 (0.62) > Arm 2 (0.35). Select **Arm 1**.
2. Suppose Arm 1 gets reward = 0 (failure). Update: Beta(4,4).
3. Second sample: Arm 1 (0.45) < Arm 2 (0.51). Select **Arm 2**.
4. Suppose Arm 2 gets reward = 1 (success). Update: Beta(3,5).

Interpretation: Thompson Sampling naturally balances exploration (uncertain arms can win by chance) and exploitation (well-performing arms have higher means and narrower distributions).

Worked Example 12: Thompson Sampling Regret Calculation

Given: 2 arms. True probabilities: $p_1 = 0.6$, $p_2 = 0.3$. After 100 pulls: Arm 1 selected 70 times, Arm 2 selected 30 times. Rewards: Arm 1 got 42 successes, Arm 2 got 9 successes.

Calculate: Cumulative regret.

Solution:

1. Optimal expected reward per pull = $p_1 = 0.6$
2. Actual expected reward = $\frac{70}{100} \times 0.6 + \frac{30}{100} \times 0.3 = 0.42 + 0.09 = 0.51$
3. Regret per pull = $0.6 - 0.51 = 0.09$
4. Cumulative regret after 100 pulls = $100 \times 0.09 = 9$

Answer: Cumulative regret = 9 (meaning the agent lost 9 units of reward compared to always pulling the optimal arm).

2.2 Linear Reward-Penalty (LR-P) Algorithm

The **Linear Reward-Penalty algorithm** is rooted in the theory of **learning automata**, which are adaptive decision-making systems. A learning automaton has a set of actions, maintains a probability distribution over them, and updates these probabilities based on environmental feedback.

2.2.1 Action Probabilities and Update Rules

Let n be the number of actions. Let $p_i(t)$ be the probability of selecting action i at time t . Initially:

$$p_i(0) = \frac{1}{n} \quad \text{for all } i$$

The update rules depend on whether the agent receives a reward or penalty:

Reward case (selected action i was good):

$$p_i(t+1) = p_i(t) + \alpha(1 - p_i(t))$$

$$p_j(t+1) = p_j(t) - \alpha p_j(t) \quad \text{for } j \neq i$$

Penalty case (selected action i was bad):

$$p_i(t+1) = p_i(t) - \beta p_i(t)$$

$$p_j(t+1) = p_j(t) + \frac{\beta p_i(t)}{n-1} \quad \text{for } j \neq i$$

where α is the **reward learning rate** and β is the **penalty learning rate**.

Worked Example 13: LR-P Probability Update – Reward Case

Given: 3 actions with probabilities $p = [0.2, 0.5, 0.3]$. Action 2 is selected and receives reward. $\alpha = 0.1$.

Calculate: Updated probabilities.

Solution:

1. $p_2(1) = p_2(0) + \alpha(1 - p_2(0)) = 0.5 + 0.1(1 - 0.5) = 0.5 + 0.05 = 0.55$
2. $p_1(1) = p_1(0) - \alpha p_1(0) = 0.2 - 0.1(0.2) = 0.2 - 0.02 = 0.18$
3. $p_3(1) = p_3(0) - \alpha p_3(0) = 0.3 - 0.1(0.3) = 0.3 - 0.03 = 0.27$

Verification: $0.18 + 0.55 + 0.27 = 1.0$. Correct.

Answer: $p = [0.18, 0.55, 0.27]$

Worked Example 14: LR-P Probability Update – Penalty Case

Given: 3 actions with probabilities $p = [0.2, 0.5, 0.3]$. Action 2 is selected and receives penalty. $\beta = 0.1$.

Calculate: Updated probabilities.

Solution:

1. $p_2(1) = p_2(0) - \beta p_2(0) = 0.5 - 0.1(0.5) = 0.5 - 0.05 = 0.45$
2. $p_1(1) = p_1(0) + \frac{\beta p_2(0)}{n-1} = 0.2 + \frac{0.1(0.5)}{2} = 0.2 + 0.025 = 0.225$
3. $p_3(1) = p_3(0) + \frac{\beta p_2(0)}{n-1} = 0.3 + 0.025 = 0.325$

Verification: $0.225 + 0.45 + 0.325 = 1.0$. Correct.

Answer: $p = [0.225, 0.45, 0.325]$

Worked Example 15: LR-P Multi-Step Convergence

Given: 2 actions, initial $p = [0.5, 0.5]$. $\alpha = 0.2$, $\beta = 0.1$. Sequence: Action 1 reward, Action 2 penalty, Action 1 reward.

Calculate: Probabilities after each step.

Solution: Step 1 (Action 1, reward):

- $p_1 = 0.5 + 0.2(1 - 0.5) = 0.5 + 0.1 = 0.6$
- $p_2 = 0.5 - 0.2(0.5) = 0.5 - 0.1 = 0.4$

Step 2 (Action 2, penalty):

- $p_2 = 0.4 - 0.1(0.4) = 0.4 - 0.04 = 0.36$
- $p_1 = 0.6 + \frac{0.1(0.4)}{1} = 0.6 + 0.04 = 0.64$

Step 3 (Action 1, reward):

- $p_1 = 0.64 + 0.2(1 - 0.64) = 0.64 + 0.072 = 0.712$
- $p_2 = 0.36 - 0.2(0.36) = 0.36 - 0.072 = 0.288$

Answer: Final $p = [0.712, 0.288]$

Interpretation: After 3 steps, the probability of selecting Action 1 has increased from 0.5 to 0.712 because it received two rewards. The algorithm converges toward favoring the action with higher reward probability.

2.3 Policy Gradient Methods

Policy gradient methods take a completely different approach from value-based methods: instead of learning a value function and deriving a policy from it, they directly learn the policy itself. The policy is represented as a parameterized function (usually a neural network) with weights θ , and the agent directly optimizes those weights to maximize expected reward using **gradient ascent**.

The key insight is that policy gradient methods can:

- Handle **continuous action spaces**, which value-based methods struggle with.
- Learn **stochastic policies** naturally, which is useful for exploration.
- Directly optimize the policy without needing an intermediate value function.

2.3.1 Objective Function

The objective function $J(\theta)$ measures the expected return when following policy π_θ :

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$$

where $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$ is a trajectory generated by the policy.

To maximize $J(\theta)$, we use gradient ascent:

$$\theta_{k+1} = \theta_k + \alpha \cdot \nabla_\theta J(\pi_{\theta_k})$$

The central challenge is computing $\nabla_\theta J(\theta)$. Since the environment is a black box, we cannot differentiate through it. The solution is the **log-derivative trick** (likelihood ratio trick).

2.3.2 The Policy Gradient Theorem

The policy gradient theorem states:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot G_t \right]$$

where G_t is the **return** (cumulative discounted reward from time t onward). The term $\nabla_\theta \log \pi_\theta(a_t | s_t)$ is called the **score function**. It points in the direction in parameter space that makes action a_t more probable in state s_t .

The policy gradient expression is derived through these steps:

1. Write $J(\theta) = \sum_\tau P(\tau; \theta) \cdot R(\tau)$ (sum over all trajectories)
2. Take gradient: $\nabla_\theta J(\theta) = \sum_\tau \nabla_\theta P(\tau; \theta) \cdot R(\tau)$
3. Apply log-derivative trick: $\nabla_\theta P(\tau; \theta) = P(\tau; \theta) \cdot \nabla_\theta \log P(\tau; \theta)$
4. Expand $\log P(\tau; \theta)$: only $\pi_\theta(a_t | s_t)$ depends on θ ; environment dynamics drop out
5. Final result: $\nabla_\theta J(\theta) = \mathbb{E}_\tau [\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot G_t]$

2.3.3 REINFORCE Algorithm

REINFORCE is the classic **Monte Carlo** policy gradient method. It estimates the policy gradient by sampling complete episodes and using the actual returns as unbiased estimates of G_t .

The algorithm steps:

1. Initialize policy parameters θ randomly.
2. Collect N episodes by following π_θ .
3. For each episode, compute returns G_t for each timestep (working backwards).
4. Compute gradient: $\nabla_\theta J \approx \frac{1}{N} \sum_i \sum_t \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \cdot G_t^i$.
5. Update: $\theta \leftarrow \theta + \alpha \cdot \nabla_\theta J$.
6. Repeat until convergence.

REINFORCE is model-free, works for both discrete and continuous action spaces, and naturally produces stochastic policies. However, it has **high variance** because it uses full-episode returns.

Worked Example 16: REINFORCE Gradient Update

Given: A simple policy with parameter θ where

$p_i^{\theta}(A|s) = \text{sigma}(\theta)$ (sigmoid). Episode: s_0

to A, $r_0 = 3$; s_1

to B, $r_1 = 5$.

$\gamma = 1$,

$\alpha = 0.1$.

$\theta = 0$ gives

$p(A|s_0) = 0.5$.

Calculate: Gradient update for θ .

Solution:

1. Compute returns: $G_0 = 3 + 1 \cdot 5 = 8$, $G_1 = 5$

2. Score function: For action A:
 $\nabla_{\theta} \log p(A|s) = 1 - p(A|s) = 1 - 0.5 = 0.5$

3. Gradient:
 $\nabla_{\theta} J = \sum_t \nabla_{\theta} \log p(a_t|s_t) \cdot G_t = 0.5 \cdot 8 + 1 \cdot 5 = 9$ (focusing on first action)

4. Update:
 $\theta_{\text{new}} = 0 + 0.1 \cdot 9 = 0.9$

Interpretation: The positive return (8) pushes the parameter in the direction that makes action A more likely in state s_0 . The magnitude of the update is proportional to the return.

Worked Example 17: REINFORCE with Discounting

Given: Episode: $(s_0, A, r_0 = 2)$

to $(s_1, B, r_1 = 4)$

to $(s_2, A, r_2 = 3)$.

$\gamma = 0.9$.

Calculate: Returns G_0, G_1, G_2 .

Solution:

1. $G_2 = r_2 = 3$ (terminal step)

2. $G_1 = r_1 + \gamma G_2 = 4 + 0.9(3) = 4 + 2.7 = 6.7$

3. $G_0 = r_0 + \gamma G_1 = 2 + 0.9(6.7) = 2 + 6.03 = 8.03$

Answer: $G_0 = 8.03, G_1 = 6.7, G_2 = 3.0$

Interpretation: Earlier actions receive larger returns because they account for all future rewards. The discount factor γ reduces the weight of distant rewards, preventing infinite returns and stabilizing learning.

Worked Example 18: REINFORCE Variance Problem

Given: Two episodes from the same policy:

- Episode 1: Returns $G = [10, 8, 6]$ (3 steps)
- Episode 2: Returns $G = [-5, -3, -1]$ (3 steps, negative reward)

For a particular state-action pair (s, a) , the score function is $\nabla_{\theta} \log \pi(a|s) = 0.3$ in both episodes.

Calculate: Gradient contribution and explain variance.

Solution:

1. Episode 1 contribution: $0.3 \times 10 = 3.0$ (for first step)
2. Episode 2 contribution: $0.3 \times (-5) = -1.5$
3. Average gradient from these two episodes:
 $\frac{3.0 + (-1.5)}{2} = 0.75$

Answer: The gradient direction is positive (toward making action a more likely in state s), but the estimate has high variance because the two episodes gave opposite-signed contributions. This is why REINFORCE often needs **baseline subtraction** (subtracting the average return) to reduce variance.

2.4 Comparison of Algorithms

Algorithm	Exploration Strategy	Best For	Key Trait
Greedy	None (exploitation only)	Baseline	Simple, gets stuck locally
ϵ -Greedy	Random with prob ϵ	Simple problems	Easy to implement
UCB	Confidence intervals	Theoretical guarantees	Deterministic, smart exploration
Thompson Sampling	Bayesian sampling	Real-world applications	Often best empirical performance
LR-P	Probability nudging	Small action spaces	Lightweight, no state memory
Policy Gradient	Gradient ascent	Continuous actions	Direct policy optimization
REINFORCE	Monte Carlo returns	Policy-based, episodic	High variance, model-free

3 Module 3: Full Reinforcement Learning and MDP

3.1 What is Full Reinforcement Learning?

Full Reinforcement Learning is the standard formulation where the agent aims to maximize the cumulative reward over the long term, considering not only immediate rewards but also future rewards that result from its actions. This contrasts with immediate RL, which only considers single-step rewards.

Three defining characteristics of full RL:

1. **Sequence of Actions:** The agent takes a chain of decisions — each action affects the future state of the environment.
2. **Delayed Reward:** Feedback is not immediate. The agent must connect past actions to future outcomes (the **credit assignment problem**).
3. **State Dependency:** The next problem (state) depends on previous actions — earlier decisions shape later opportunities.

3.2 Full RL vs Immediate RL

Aspect	Immediate RL	Full RL
Reward horizon	Single step	Long-term, cumulative
Planning	Not required	Essential
Credit assignment	None	Central challenge
Examples	Coin collection, ad clicks	Chess, robotics, maze navigation

3.3 Agent–Environment Interface

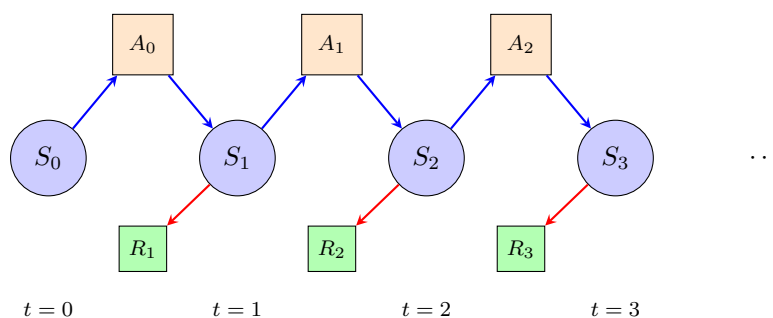
At each discrete time step $t = 0, 1, 2, 3, \dots$:

1. Agent observes state $S_t \in \mathcal{S}$
2. Agent selects action $A_t \in \mathcal{A}(S_t)$
3. Environment gives reward $R_{t+1} \in \mathbb{R}$ and new state S_{t+1}

This produces the trajectory:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, S_3, \dots$$

Markov Decision Process - Trajectory Visualization



The Credit Assignment Problem

In Full RL, rewards may be delayed many steps after the actions that caused them. This creates the **credit assignment problem**: how to attribute success/failure to individual actions?

Example: In chess, a brilliant opening move might contribute to a win 50 moves later. The agent must propagate credit backward through time.

Mathematical Formulation: The return G_t incorporates all future rewards, weighted by discount factor:

$$G_t = \underbrace{R_{t+1}}_{\text{immediate}} + \underbrace{\gamma R_{t+2} + \gamma^2 R_{t+3} + \dots}_{\text{delayed}}$$

TD learning and Monte Carlo methods differ in how they solve credit assignment:

- **Monte Carlo:** Wait until episode ends, then assign full return to all visited states
- **TD Learning:** Use bootstrapping — estimate credit immediately based on next state's value

3.4 Goals and Rewards

The **Reward Hypothesis** (Sutton & Barto): “All of what we mean by goals and purposes can be well thought of as the maximisation of the expected value of the cumulative sum of a received scalar signal (reward).”

Characteristics of a good reward signal:

- **Scalar:** A single number — simple for learning algorithms to process.
- **Outside Agent's Control:** The agent cannot manipulate the reward signal itself. Forces honest behavior.
- **Bounded:** Rewards are finite. Prevents value estimates from diverging to $\pm\infty$.

The reward signal tells the agent **what** to achieve, not **how**. The agent must discover effective behavior on its own.

3.5 Returns and Discounting

The agent seeks to maximize the expected **return** G_t .

For **episodic tasks** (finite horizon T):

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

For **continuing tasks** (infinite horizon):

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where $\gamma \in [0, 1]$ is the **discount factor**.

Key properties:

- $\gamma = 0$: Myopic agent — only cares about immediate reward R_{t+1} .
- $\gamma \rightarrow 1$: Farsighted agent — treats distant future rewards almost as valuable as immediate ones.
- $\gamma < 1$ guarantees convergence of the infinite sum if rewards are bounded.

A useful recursive identity:

$$G_t = R_{t+1} + \gamma G_{t+1}$$

3.5.1 Unified Notation

Using the **absorbing state trick**: model episode termination as entering a special absorbing state that only transitions to itself with $R = 0$. Now both episodic and continuing tasks use the same formula:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

with the constraint that we cannot have both $T = \infty$ and $\gamma = 1$ simultaneously (otherwise the sum diverges).

3.6 The Markov Property

The **Markov property** is the foundation of RL state representation:

$$P(S_{t+1}|S_t, A_t) = P(S_{t+1}|S_0, A_0, \dots, S_t, A_t)$$

In words: “The future is independent of the past, given the present.” The next state depends **only** on the current state and action. The full history before time t is irrelevant if S_t summarizes everything needed.

This allows algorithms to use **one-step transitions** for learning — no need to store and process the full history of past states.

If the state does not capture enough information, the Markov property fails, leading to **Partially Observable MDPs (POMDPs)**.

3.7 Episodic vs Continuing Tasks

Episodic tasks naturally break into episodes — finite sequences that each end at a terminal state, after which the environment resets. Examples: chess games, maze solving attempts, pole balancing episodes.

Continuing tasks have no terminal state. The agent–environment interaction goes on indefinitely. Examples: personal assistant robots, process control in factories. Rewards must be discounted to keep the return finite.

3.8 State Value Function $V_\pi(s)$

The **state-value function** $v_\pi(s)$ gives the expected return when starting in state s and following policy π thereafter:

$$v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s \right]$$

Key insight: “The value of a state is not a fixed property of the state — it is a property of the state AND the policy.” The same state can have different values under different policies.

3.9 Action Value Function $Q_\pi(s, a)$

The **action-value function** $q_\pi(s, a)$ gives the expected return when starting in state s , taking action a , and thereafter following policy π :

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s, A_t = a \right]$$

3.10 Relationship Between V and Q

The fundamental relationship:

$$V_\pi(s) = \sum_a \pi(a|s) \cdot Q_\pi(s, a)$$

The state value is a **weighted average** of action values, where the weights are given by the policy probabilities. This means if we know $Q_\pi(s, a)$ for all actions, we can compute $V_\pi(s)$.

Conversely:

$$Q_\pi(s, a) = \sum_{s'} P(s'|s, a) [r + \gamma V_\pi(s')]$$

The action value depends on the immediate reward and the discounted value of the next state.

Worked Example 19: Computing $V(0)$ Under Different Policies

Given: A 2-state MDP. From state 0: going left gives $r = +1$; going right gives $r = +2$. Episode ends after one action. $\gamma = 1$. Fixed Q-values: $Q(0, \text{left}) = 1.0$, $Q(0, \text{right}) = 2.0$.

Calculate: $V(0)$ for four policies.

Solution: Using $V_\pi(0) = \pi(\text{left}|0) \cdot Q(0, \text{left}) + \pi(\text{right}|0) \cdot Q(0, \text{right})$:

1. **Always Left:** $\pi = [1.0, 0.0]$

$$V(0) = 1.0(1.0) + 0.0(2.0) = 1.0$$

2. **Always Right:** $\pi = [0.0, 1.0]$

$$V(0) = 0.0(1.0) + 1.0(2.0) = 2.0$$

3. **50/50:** $\pi = [0.5, 0.5]$

$$V(0) = 0.5(1.0) + 0.5(2.0) = 0.5 + 1.0 = 1.5$$

4. **20% Left / 80% Right:** $\pi = [0.2, 0.8]$

$$V(0) = 0.2(1.0) + 0.8(2.0) = 0.2 + 1.6 = 1.8$$

Answer: $V(0) = 1.0, 2.0, 1.5, 1.8$ for the four policies.

Interpretation: The value of state 0 changes purely based on the policy, even though the Q-values are fixed. The optimal policy is “Always Right” with $V(0) = 2.0$.

Worked Example 20: Value Function with Future Consequences

Given: 4-state world: $0 \xrightarrow{\text{left}} 1$ ($r = +1$), $0 \xrightarrow{\text{right}} 2$ ($r = +2$), $2 \xrightarrow{\text{right}} 3$ ($r = -10$). No further transitions. $\gamma = 1$.

Calculate: Updated Q-values and $V(0)$ under each policy.

Solution:

1. $Q(0, \text{left}) = +1.0$ (unchanged, ends at state 1)
2. $Q(0, \text{right}) = +2 + (-10) = -8.0$ (goes to state 2, then penalty at state 3)

Policies:

1. Always Left: $V(0) = 1.0(-8.0) + 1.0(1.0) = 1.0$
2. Always Right: $V(0) = 1.0(-8.0) = -8.0$
3. 50/50: $V(0) = 0.5(1.0) + 0.5(-8.0) = 0.5 - 4.0 = -3.5$
4. 20/80: $V(0) = 0.2(1.0) + 0.8(-8.0) = 0.2 - 6.4 = -6.2$

Answer: Always Left now gives $V(0) = 1.0$ (best); Always Right gives $V(0) = -8.0$ (catastrophic).

Key Insight: Maximizing immediate reward can destroy long-term return. In the 2-state world, “Always Right” was optimal. In the 4-state world, it is catastrophic because the immediate +2 is followed by a -10 penalty. This is exactly why full RL is needed: TD-learning and Q-learning discover $Q(0, \text{right}) = -8$ by propagating future penalties backward in time.

Worked Example 21: Discounted Return Calculation

Given: Episode rewards: $R_1 = 2$, $R_2 = 3$, $R_3 = 5$, $R_4 = 1$. $\gamma = 0.9$.

Calculate: Returns G_0 , G_1 , G_2 , G_3 .

Solution:

1. $G_3 = R_4 = 1$
2. $G_2 = R_3 + \gamma G_3 = 5 + 0.9(1) = 5.9$
3. $G_1 = R_2 + \gamma G_2 = 3 + 0.9(5.9) = 3 + 5.31 = 8.31$
4. $G_0 = R_1 + \gamma G_1 = 2 + 0.9(8.31) = 2 + 7.479 = 9.479$

Answer: $G_0 = 9.48$, $G_1 = 8.31$, $G_2 = 5.9$, $G_3 = 1.0$

Interpretation: With $\gamma = 0.9$, distant rewards are weighted slightly less. If $\gamma = 0$, all returns would equal the immediate next reward. If $\gamma = 1$, $G_0 = 2 + 3 + 5 + 1 = 11$.

3.11 Bellman Equations

The **Bellman equations** express values recursively — V and Q are defined in terms of each other. They are the foundation of dynamic programming and reinforcement learning algorithms.

3.11.1 Bellman Equation for V_π

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [r + \gamma v_\pi(s')]$$

Meaning: The value of state s equals the expected immediate reward plus the discounted value of the next state, averaged over all actions (weighted by π) and all possible next states.

3.11.2 Bellman Equation for Q_π

$$q_\pi(s, a) = \sum_{s'} P(s'|s, a) [r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s', a')]$$

3.11.3 Bellman Optimality Equations

For the **optimal** value functions:

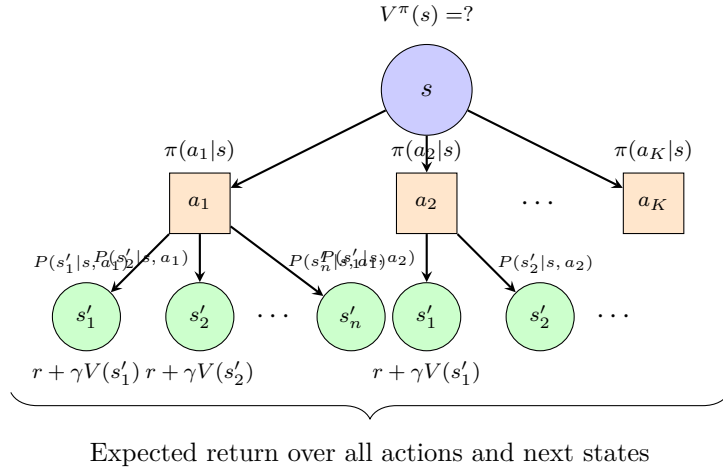
$$v^*(s) = \max_a \sum_{s'} P(s'|s, a) [r + \gamma v^*(s')]$$

$$q^*(s, a) = \sum_{s'} P(s'|s, a) [r + \gamma \max_{a'} q^*(s', a')]$$

The optimal policy is then:

$$\pi^*(s) = \arg \max_a q^*(s, a)$$

Bellman Backup Diagram for $V^\pi(s)$



Bellman Equations: Theoretical Foundations

Key Insight: The Bellman equations encode the principle of **optimality** — an optimal policy has the property that whatever the initial state and decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision (Bellman, 1957).

Contraction Mapping: The Bellman backup operator $\mathcal{T}^\pi$ is a **contraction** in the sup-norm:

$$\|\mathcal{T}^\pi V - \mathcal{T}^\pi V'\|_\infty \leq \gamma \|V - V'\|_\infty$$

This guarantees:

1. **Unique solution:** There exists exactly one value function v_π satisfying the Bellman equation
2. **Convergence:** Iterative policy evaluation converges to v_π
3. **Rate:** Convergence is geometric with factor γ

Proof Sketch:

$$|\mathcal{T}^\pi V(s) - \mathcal{T}^\pi V'(s)| = \left| \gamma \sum_{s'} P(s'|s, a) [V(s') - V'(s')] \right| \quad (1)$$

$$\leq \gamma \sum_{s'} P(s'|s, a) |V(s') - V'(s')| \quad (2)$$

$$\leq \gamma \|V - V'\|_\infty \quad (3)$$

Worked Example: Bellman Operator Application

Given: A simple 2-state MDP with deterministic transitions. Policy: always go to the other state. $\gamma = 0.9$. Initial value guess: $V_0 = [0, 0]$.

Apply the Bellman backup operator $\mathcal{T}^\pi$:

Iteration 1:

$$V_1(1) = r + \gamma V_0(2) = 1 + 0.9(0) = 1$$

$$V_1(2) = r + \gamma V_0(1) = 1 + 0.9(0) = 1$$

Result: $V_1 = [1, 1]$

Iteration 2:

$$V_2(1) = 1 + 0.9(1) = 1.9$$

$$V_2(2) = 1 + 0.9(1) = 1.9$$

Result: $V_2 = [1.9, 1.9]$

Iteration k :

$$V_k = \sum_{i=0}^{k-1} \gamma^i = \frac{1 - \gamma^k}{1 - \gamma}$$

Limit ($k \rightarrow \infty$):

$$V_\infty = \frac{1}{1 - \gamma} = \frac{1}{0.1} = 10$$

This matches the analytical solution from the Bellman equation!

Worked Example 22: Bellman Equation – Recycling Robot

Given: A recycling robot with two energy levels: High (h) and Low (l). Actions: search (s), wait (w), recharge (re). Transitions:

- Search when High: stays High with prob α , becomes Low with prob $1 - \alpha$
- Search when Low: stays Low with prob β , battery dies with prob $1 - \beta$ (rescue required)
- Recharge: goes to High with probability 1
- Rewards: r_s (search), r_w (wait), -3 (rescue)

Write: Bellman optimality equations for $v^*(h)$ and $v^*(l)$.

Solution: For state High:

$$v^*(h) = \max \begin{cases} r_s + \gamma[\alpha v^*(h) + (1 - \alpha)v^*(l)] & \text{(search)} \\ r_w + \gamma v^*(h) & \text{(wait)} \end{cases}$$

For state Low:

$$v^*(l) = \max \begin{cases} r_s + \gamma[\beta v^*(l) + (1 - \beta)(-3)] & \text{(search)} \\ r_w + \gamma v^*(l) & \text{(wait)} \\ \gamma v^*(h) & \text{(recharge)} \end{cases}$$

Interpretation: At each state, the optimal value is the maximum over all possible actions. Each action's value is the immediate reward plus the discounted expected value of the next state(s). The equations are coupled — $v^*(h)$ depends on $v^*(l)$ and vice versa.

Worked Example 23: Solving a Simple Bellman Equation

Given: A 2-state MDP with deterministic transitions. $r = +1$ for any transition. $\gamma = 0.9$. Policy: always go to the other state. Find $v_\pi(1)$ and $v_\pi(2)$.

Solution:

1. $v_\pi(1) = 1 + \gamma v_\pi(2) = 1 + 0.9v_\pi(2)$
2. $v_\pi(2) = 1 + \gamma v_\pi(1) = 1 + 0.9v_\pi(1)$
3. Substitute (2) into (1): $v_\pi(1) = 1 + 0.9(1 + 0.9v_\pi(1)) = 1 + 0.9 + 0.81v_\pi(1)$
4. $v_\pi(1) - 0.81v_\pi(1) = 1.9$
5. $0.19v_\pi(1) = 1.9$
6. $v_\pi(1) = \frac{1.9}{0.19} = 10.0$
7. $v_\pi(2) = 1 + 0.9(10) = 1 + 9 = 10.0$

Answer: $v_\pi(1) = 10.0$, $v_\pi(2) = 10.0$

Interpretation: Due to symmetry, both states have the same value. The high value (10) comes from the infinite horizon with positive rewards and $\gamma < 1$ ensuring convergence.

Worked Example 24: Optimal Policy Selection from Q-Values

Given: Q-values at state s : $Q(s, a_1) = 5.2$, $Q(s, a_2) = 7.8$, $Q(s, a_3) = 3.1$.

Calculate: Optimal action and $V^*(s)$.

Solution:

1. Optimal action: $\pi^*(s) = \arg \max_a Q(s, a) = a_2$ (since $7.8 > 5.2 > 3.1$)

2. $V^*(s) = \max_a Q(s, a) = 7.8$

Answer: $\pi^*(s) = a_2$, $V^*(s) = 7.8$

Interpretation: The optimal policy is greedy with respect to the optimal Q-function. This is why Q-learning is so powerful: once we learn Q^* , the optimal policy is trivial to extract.

4 Module 4: Dynamic Programming and Monte Carlo Methods

4.1 Dynamic Programming in RL

Dynamic Programming (DP) refers to a collection of algorithms that can compute optimal policies for a Markov Decision Process (MDP) given a **perfect model** of the environment. A perfect model means both **transition probabilities** and **reward model** are known for the entire environment.

The key idea behind DP is the use of **value functions** to organize and structure the search for good policies. DP algorithms turn Bellman equations into **update rules** for improving approximations of the desired functions. DP contains two main steps:

1. Break the problem into subproblems and solve each one.
2. Cache or store solutions to subproblems for reuse to find the overall optimal solution.

To solve a given MDP, we need to:

1. Find out how good an arbitrary policy is (policy evaluation).
2. Find out the optimal policy for the given MDP (policy improvement).

Limitations:

- Assumption of a perfect model is often unavailable for most real-world problems.
- Computational expense is very high — it does not scale well as the number of states increases.
- However, DP is useful as a theoretical foundation on which most advanced RL methods are based.

4.2 Policy Evaluation (Prediction)

Policy evaluation computes the state-value function v_π for an arbitrary policy π . Starting with some random initialization v_0 , we iteratively update using the Bellman equation for v_π :

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_k(s')]$$

This is called **Iterative Policy Evaluation (IPE)**. Each sweep updates all states. The sequence $v_0, v_1, v_2, \dots$ converges to v_π as $k \rightarrow \infty$.

4.2.1 Visual Gridworld Reference

All gridworld examples use the standard 4×4 gridworld (Sutton & Barto). States 1–16 in row-major order. States 1 and 16 are terminal with $v = 0$. All moves give $r = -1$. Actions **up**, **down**, **left**, **right** are deterministic; boundary moves keep the agent in place.

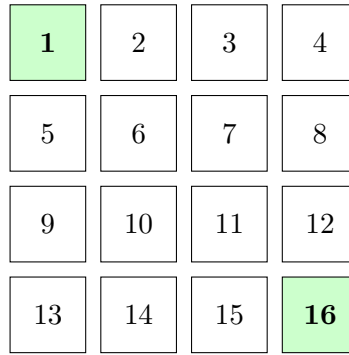


Figure 1: Standard 4×4 gridworld. Green cells are terminal (value 0). All moves incur $r = -1$.

State 6 (center) is the focal state. Neighbors: up $\rightarrow$ 2, down $\rightarrow$ 10, left $\rightarrow$ 5, right $\rightarrow$ 7. The visual diagrams below overlay the Bellman backup onto this layout.

Worked Example 25: Gridworld Policy Evaluation

Given: A 4x4 gridworld with states numbered 1–16. States 1 and 16 are terminal (absorbing) with value 0. All other transitions give reward $r = -1$. Discount factor $\gamma = 1$. Actions: up, down, left, right (deterministic). Policy: **random** — each action has probability 0.25. Compute v_1 for state 6.

Solution: State 6 is surrounded by states 2 (up), 10 (down), 5 (left), 7 (right). All non-terminal.

For the random policy, each direction has probability 0.25. The Bellman update for state 6:

$$v_1(6) = \sum_a \pi(a|6) \sum_{s'} p(s'|6, a) [r + \gamma v_0(s')]$$

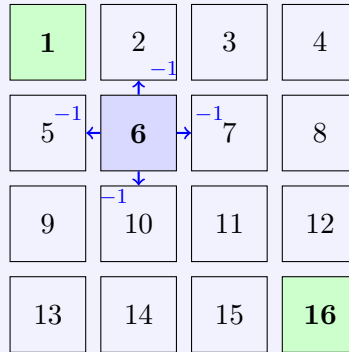
With $v_0(s) = 0$ for all non-terminal states and $v_0(1) = v_0(16) = 0$:

1. Up (to state 2): $0.25 \times (-1 + 1 \cdot 0) = -0.25$
2. Down (to state 10): $0.25 \times (-1 + 1 \cdot 0) = -0.25$
3. Left (to state 5): $0.25 \times (-1 + 1 \cdot 0) = -0.25$
4. Right (to state 7): $0.25 \times (-1 + 1 \cdot 0) = -0.25$

$$v_1(6) = -0.25 - 0.25 - 0.25 - 0.25 = -1.0$$

Answer: $v_1(6) = -1.0$

Visual Backup at State 6 (v_0 initialization):



State 6 receives four -1 rewards, each weighted 0.25.

Similarly, for all non-terminal states adjacent to non-terminal states: $v_1(s) = -1$.

For states adjacent to terminal states (e.g., state 2 adjacent to state 1):

- Up (to state 1, terminal): $0.25 \times (-1 + 1 \cdot 0) = -0.25$
- Other directions: -0.25 each
- $v_1(2) = -1.0$ as well (all directions give -1 reward)

Interpretation: After one sweep, every non-terminal state has value -1 , reflecting that any action incurs a -1 penalty. As sweeps continue, values propagate outward from the terminal states.

Worked Example 26: Gridworld Second Sweep (v_2)

Given: Same gridworld. After first sweep, $v_1(s) = -1$ for all non-terminal states, $v_1(1) = v_1(16) = 0$. Compute $v_2(6)$.

Solution:

1. Up (to state 2): $0.25 \times (-1 + 1 \cdot (-1)) = 0.25 \times (-2) = -0.5$
2. Down (to state 10): $0.25 \times (-1 + (-1)) = -0.5$
3. Left (to state 5): $0.25 \times (-1 + (-1)) = -0.5$
4. Right (to state 7): $0.25 \times (-1 + (-1)) = -0.5$

$$v_2(6) = -0.5 - 0.5 - 0.5 - 0.5 = -2.0$$

Answer: $v_2(6) = -2.0$

Visual Value Map after Sweep 1 (v_1 in each cell):

0	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	0

The highlighted cell (state 6) is the focus. All non-terminal cells show $v_1 = -1$ because every move incurs -1 and neighbors still had $v_0 = 0$. Terminal cells remain 0 (green).

For states adjacent to terminal states, some directions lead to terminal (value 0):

- $v_2(2)$: Up (terminal): $0.25(-1 + 0) = -0.25$; other 3 directions: $0.25(-1 - 1) = -0.5$ each
- $v_2(2) = -0.25 - 0.5 - 0.5 - 0.5 = -1.75$

Interpretation: Values propagate as negative distances from terminal states. After many sweeps, values converge to the expected number of steps to reach a terminal state.

Worked Example 27: Value Iteration Step

Given: Same gridworld, but now with an **optimal** policy. We use value iteration (Bellman optimality backup). Compute $v_{k+1}(6)$ given $v_k(6) = -2$, $v_k(2) = v_k(10) = v_k(5) = v_k(7) = -2$.

Solution: Value iteration takes the **maximum** over actions:

$$v_{k+1}(6) = \max_a \sum_{s'} p(s'|6, a) [r + \gamma v_k(s')]$$

For each action from state 6:

- Up (to 2): $-1 + (-2) = -3$
- Down (to 10): $-1 + (-2) = -3$
- Left (to 5): $-1 + (-2) = -3$
- Right (to 7): $-1 + (-2) = -3$

All actions give -3 , so the maximum is -3 .

Answer: $v_{k+1}(6) = -3$

Visual Value Iteration Step: We take the **max** over actions. The diagram shows v_k values (all -2); all neighbors are equal so every action yields -3 . In later sweeps the max will point toward the nearest terminal.

0	-2	-2	-2
-2	-2	-2	-2
-2	-2	-2	-2
-2	-2	-2	0

Interpretation: In this symmetric case, all actions are equally bad. As values converge, the agent learns which paths lead to the nearest terminal state. Value iteration converges faster than full policy iteration because it combines evaluation and improvement in one step.

4.3 Policy Improvement

Once we know v_π , we can improve the policy by making it greedy with respect to v_π . The new policy π' selects at each state the action that maximizes the expected value:

$$\pi'(s) = \arg \max_a \sum_{s', r} p(s', r|s, a) [r + \gamma v_\pi(s')]$$

The **policy improvement theorem** guarantees that π' is at least as good as π , and strictly better if any action is strictly better than $\pi(s)$.

4.4 Policy Iteration

Policy iteration alternates between policy evaluation and policy improvement:

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \dots \xrightarrow{I} \pi^*$$

Each iteration is guaranteed to produce a policy that is at least as good as the previous one. Given enough iterations, it converges to the optimal policy π^* .

A drawback is that each policy evaluation step itself requires multiple sweeps through all states. **Value iteration** addresses this by stopping evaluation after just one sweep, combining evaluation and improvement into a single update.

4.5 Value Iteration

Value iteration directly applies the Bellman optimality equation as an update rule:

$$v_{k+1}(s) = \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v_k(s')]$$

This is identical to the Bellman update in policy evaluation, except we take the **maximum** over all actions instead of averaging over the policy. Once the updates are small enough (below a threshold θ), we can extract the optimal policy by acting greedily with respect to the final value function.

4.6 Bootstrapping

Bootstrapping means estimating something based on another estimation. In DP methods such as value iteration and policy iteration, bootstrapping is used to update the value function of a state based on the **estimated value of successor states**. This allows for efficient calculation of the optimal value function and policy.

4.7 Monte Carlo Methods

Monte Carlo (MC) methods require only **experience** — sample sequences of states, actions, and rewards from actual or simulated interaction with an environment. The name comes from the Monte Carlo casino — randomness and chance are central to the method. The core idea: instead of computing something analytically, you run random experiments and average the results.

Key difference from DP: MC methods do **not** require a model of the environment. They learn directly from sampled episodes.

4.7.1 Types of Monte Carlo Prediction

First-Visit Monte Carlo: Estimates $V(s)$ as the average of returns following the **first visit** to state s in each episode.

Every-Visit Monte Carlo: Estimates $V(s)$ as the average of returns following **every visit** to state s in each episode.

Worked Example 28: First-Visit vs Every-Visit MC

Given: Two episodes:

- Episode 1: $A \xrightarrow{+3} A \xrightarrow{+2} B \xrightarrow{-4} B \xrightarrow{+4} A \xrightarrow{-3}$ (terminate)
- Episode 2: $A \xrightarrow{+3} B \xrightarrow{-2} B \xrightarrow{+3} A \xrightarrow{-3}$ (terminate)

Calculate: $V(A)$ and $V(B)$ using First-Visit and Every-Visit MC. $\gamma = 1$.

Solution — First-Visit MC:

1. **For** $V(A)$:

- Episode 1, first visit to A: return = $3 + 2 - 4 + 4 - 3 = 2$
- Episode 2, first visit to A: return = $3 - 2 + 3 - 3 = 1$
- $V(A) = \frac{2+1}{2} = 1.5$

2. **For** $V(B)$:

- Episode 1, first visit to B: return = $-4 + 4 - 3 = -3$
- Episode 2, first visit to B: return = $-2 + 3 - 3 = -2$
- $V(B) = \frac{-3+(-2)}{2} = -2.5$

Solution — Every-Visit MC:

1. **For** $V(A)$:

- Episode 1 visits to A: first A gives $3 + 2 - 4 + 4 - 3 = 2$; second A gives $4 - 3 = 1$
- Episode 2 visits to A: first A gives $3 - 2 + 3 - 3 = 1$
- $V(A) = \frac{2+1+1}{3} = \frac{4}{3} \approx 1.33$

2. **For** $V(B)$:

- Episode 1 visits to B: first B gives $-4 + 4 - 3 = -3$; second B gives -3
- Episode 2 visits to B: first B gives $-2 + 3 - 3 = -2$; second B gives -3
- $V(B) = \frac{-3+(-3)+(-2)+(-3)}{4} = \frac{-11}{4} = -2.75$

Answer:

	First-Visit	Every-Visit
$V(A)$	1.5	1.33
$V(B)$	-2.5	-2.75

Interpretation: First-Visit and Every-Visit give different estimates. In the limit of infinite episodes, both converge to the true value. First-Visit is unbiased; Every-Visit is biased in finite samples but still consistent. For Blackjack (where states never repeat within an episode), both methods are identical.

Worked Example 29: Monte Carlo Control

Given: Two episodes with exploring starts. After episode 1, $Q(s_1, a_1) = 5$ (1 sample). Episode 2 visits (s_1, a_1) with return $G = 7$.

Calculate: Updated $Q(s_1, a_1)$ using incremental MC update. $\alpha = 0.1$.

Solution:

1. First-visit count for (s_1, a_1) becomes 2.
2. Incremental update: $Q_{\text{new}} = Q_{\text{old}} + \frac{1}{N}(G - Q_{\text{old}})$
3. $Q_{\text{new}} = 5 + \frac{1}{2}(7 - 5) = 5 + 1 = 6$

Or with constant step-size: $Q_{\text{new}} = 5 + 0.1(7 - 5) = 5 + 0.2 = 5.2$

Answer: $Q = 6$ (sample average) or $Q = 5.2$ (constant step-size)

Interpretation: Sample average converges to the true mean. Constant step-size adapts faster to non-stationary environments but never fully converges — it keeps oscillating around the true value.

Worked Example 30: Blackjack State Value

Given: In Blackjack, the state is (player sum, dealer showing, usable ace). Consider the policy: stick if sum is 20 or 21, otherwise hit. After 1000 episodes, the average return following state (sum=18, dealer=7, no ace) is -0.15 .

Calculate: $V(\text{sum} = 18, \text{dealer} = 7, \text{no ace})$.

Solution:

1. Under first-visit MC: average of returns from first visits to this state across all episodes.
2. $V = -0.15$

Answer: $V = -0.15$

Interpretation: A slightly negative value indicates this state is slightly unfavorable under the current policy. The player is likely to bust or get a low final sum. Monte Carlo averages over the stochasticity of card draws, giving a reliable estimate given enough samples.

4.8 On-Policy vs Off-Policy Learning

On-Policy learning: The algorithm evaluates and improves the **same** policy that is being used to select actions. Examples: Policy Iteration, Value Iteration, SARSA.

Off-Policy learning: The agent learns the value of a **target policy** π while following a separate **behavior policy** b for exploration. This allows continuous exploration while learning the optimal policy. Examples: Q-Learning.

Benefits of off-policy methods:

- **Continuous exploration:** The behavior policy can keep exploring while the target policy converges to optimal.
- **Learning from demonstration:** Can learn from human demonstrations or other agents.
- **Parallel learning:** Multiple behavior policies can generate data for a single target policy.

5 Module 5: Temporal Difference Learning

5.1 What is Temporal Difference Learning?

Temporal Difference (TD) learning is a foundational class of algorithms in reinforcement learning that combine ideas from **Monte Carlo** methods and **Dynamic Programming**. Like MC, TD

learns directly from raw experience without a model. Like DP, TD **bootstraps** — updates guesses based on other guesses.

The key TD learning algorithm is **TD(0)**, which updates the value function after every step using the observed reward and the estimated value of the next state:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

The term $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is called the **TD error**. It represents the difference between:

- The **bootstrap estimate** of the return: $R_{t+1} + \gamma V(S_{t+1})$ — “what I think the return is, based on immediate reward plus discounted estimate of next state”
- The **current estimate**: $V(S_t)$ — “what I currently believe this state’s value is”

If $\delta_t > 0$, the outcome was better than expected — we should **increase** $V(S_t)$. If $\delta_t < 0$, the outcome was worse than expected — we should **decrease** $V(S_t)$. The learning rate α controls how aggressively we update.

Advantages of TD(0):

- Online learning: updates after every step, unlike MC which waits for episode end.
- Works in continuing (non-episodic) environments.
- Usually converges faster than MC in practice.

Worked Example 31: TD(0) Update

Given: $\alpha = 0.1$, $\gamma = 0.9$. Current estimates: $V(S_1) = 5.0$, $V(S_2) = 7.0$. Agent takes action in S_1 , receives $R_{t+1} = 2$, and transitions to S_2 .

Calculate: Updated $V(S_1)$.

Solution:

1. Compute TD target: $R_{t+1} + \gamma V(S_2) = 2 + 0.9(7.0) = 2 + 6.3 = 8.3$
2. Compute TD error: $\delta_t = 8.3 - 5.0 = 3.3$
3. Update: $V(S_1) = 5.0 + 0.1(3.3) = 5.0 + 0.33 = 5.33$

Answer: $V(S_1) = 5.33$

Interpretation: The immediate reward plus the value of the next state (8.3) was higher than our current estimate (5.0), so we increased $V(S_1)$ toward the TD target. With more updates, $V(S_1)$ will converge to its true value.

Worked Example 32: TD(0) Negative Update

Given: $\alpha = 0.2$, $\gamma = 1.0$. Current: $V(A) = 10.0$, $V(B) = 8.0$, $V(C) = 5.0$. Agent moves $A \rightarrow B$ with $R = -1$, then $B \rightarrow C$ with $R = -2$.

Calculate: $V(A)$ and $V(B)$ after two TD(0) updates.

Solution: Step 1: $A \rightarrow B$, $R = -1$

1. TD target for A: $R + \gamma V(B) = -1 + 1.0(8.0) = 7.0$
2. TD error: $\delta = 7.0 - 10.0 = -3.0$
3. $V(A) = 10.0 + 0.2(-3.0) = 10.0 - 0.6 = 9.4$

Step 2: $B \rightarrow C$, $R = -2$

1. TD target for B: $R + \gamma V(C) = -2 + 1.0(5.0) = 3.0$
2. TD error: $\delta = 3.0 - 8.0 = -5.0$
3. $V(B) = 8.0 + 0.2(-5.0) = 8.0 - 1.0 = 7.0$

Answer: $V(A) = 9.4$, $V(B) = 7.0$

Interpretation: Both states were overestimated. The negative rewards and lower-valued successor states caused the TD errors to be negative, reducing the value estimates. With more episodes, these estimates converge to the true expected returns.

Worked Example 33: TD(0) Over Multiple Episodes

Given: A 2-state MDP: Start $\rightarrow A$ ($R = 0$) $\rightarrow B$ ($R = 10$), then terminate. $\gamma = 0.9$, $\alpha = 0.5$. Initial: $V(A) = 0$, $V(B) = 0$.

Episode 1: returns 10 at B. Episode 2: same. Episode 3: same.

Calculate: $V(A)$ and $V(B)$ after 3 episodes.

Solution: Episode 1:

1. $A \rightarrow B$: TD target $= 0 + 0.9(0) = 0$, error $= 0 - 0 = 0$. $V(A) = 0$.
2. $B \rightarrow$ terminal: TD target $= 10 + 0.9(0) = 10$, error $= 10 - 0 = 10$. $V(B) = 0 + 0.5(10) = 5$.

Episode 2:

1. $A \rightarrow B$: TD target $= 0 + 0.9(5) = 4.5$, error $= 4.5 - 0 = 4.5$. $V(A) = 0 + 0.5(4.5) = 2.25$.
2. $B \rightarrow$ terminal: TD target $= 10 + 0 = 10$, error $= 10 - 5 = 5$. $V(B) = 5 + 0.5(5) = 7.5$.

Episode 3:

1. $A \rightarrow B$: TD target $= 0 + 0.9(7.5) = 6.75$, error $= 6.75 - 2.25 = 4.5$. $V(A) = 2.25 + 0.5(4.5) = 4.5$.
2. $B \rightarrow$ terminal: TD target $= 10$, error $= 10 - 7.5 = 2.5$. $V(B) = 7.5 + 0.5(2.5) = 8.75$.

Answer: After 3 episodes: $V(A) = 4.5$, $V(B) = 8.75$

Interpretation: $V(B)$ converges toward 10 (the true terminal reward). $V(A)$ converges toward $0.9 \times 10 = 9$ (the discounted reward from reaching B). TD bootstraps — $V(A)$ improves as $V(B)$ improves, even without reaching the terminal state.

5.2 On-Policy vs Off-Policy TD Control

On-policy: Learn about the policy being followed (behavior policy = target policy). The agent explores and learns the same policy simultaneously. **SARSA** is an on-policy TD control algorithm.

Off-policy: Learn about a policy different from the one being followed (behavior policy $\neq$ target

policy). The agent explores with one policy but learns the optimal target policy. **Q-Learning** is an off-policy TD control algorithm.

5.3 SARSA: On-Policy TD Control

SARSA updates the Q-value after every transition $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$ — hence the name SARSA. The update rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

The next action A_{t+1} is selected using the **current** policy (e.g., ϵ -greedy with respect to current Q). This makes SARSA **on-policy**.

Worked Example 34: SARSA Update

Given: $\alpha = 0.5$, $\gamma = 1.0$. Q-table:

	Left	Right	Up
A	2.0	1.0	0.5
B	1.5	3.0	2.0
C	0.5	1.0	2.5

Agent in state A takes action Right, gets $R = 5$, moves to B. In state B, uses greedy action (Up, $Q = 2.0$).

Calculate: Updated $Q(A, \text{Right})$.

Solution:

1. Current $Q(A, \text{Right}) = 1.0$
2. SARSA target: $R + \gamma Q(B, \text{Up}) = 5 + 1.0(2.0) = 7.0$
3. TD error: $7.0 - 1.0 = 6.0$
4. Update: $Q(A, \text{Right}) = 1.0 + 0.5(6.0) = 1.0 + 3.0 = 4.0$

Answer: $Q(A, \text{Right}) = 4.0$

Interpretation: The transition from A to B via Right was very rewarding (immediate reward 5 + future value 2). SARSA updates $Q(A, \text{Right})$ aggressively toward this target.

Worked Example 35: SARSA With Exploration

Given: Same Q-table as Example 34. $\epsilon = 0.1$. Agent in state A takes action Right (greedy), gets $R = 5$, moves to B. In B, ϵ -greedy randomly selects Left ($Q = 1.5$) instead of greedy Up.

Calculate: Updated $Q(A, \text{Right})$.

Solution:

1. Current $Q(A, \text{Right}) = 1.0$
2. SARSA target: $R + \gamma Q(B, \text{Left}) = 5 + 1.0(1.5) = 6.5$
3. TD error: $6.5 - 1.0 = 5.5$
4. Update: $Q(A, \text{Right}) = 1.0 + 0.5(5.5) = 1.0 + 2.75 = 3.75$

Answer: $Q(A, \text{Right}) = 3.75$

Key Difference from Q-Learning: SARSA uses the actual next action taken (Left, due to exploration), giving a target of 6.5. Q-Learning would use the maximum next Q-value (Up = 2.0), giving a target of 7.0. SARSA learns a more **conservative** policy that accounts for exploration mistakes.

Worked Example 36: SARSA Episode Walkthrough

Given: A 2-state chain: $A \rightarrow B \rightarrow \text{terminal}$. Actions: only one action per state. $R(A) = 0$, $R(B) = 10$. $\gamma = 0.9$, $\alpha = 0.5$. Initial $Q(A) = 0$, $Q(B) = 0$.

Episode: $A \rightarrow B$ ($R=0$), $B \rightarrow \text{terminal}$ ($R=10$).

Calculate: Updated Q-values.

Solution:

1. In A: target = $0 + 0.9 \cdot Q(B) = 0 + 0 = 0$. Error = $0 - 0 = 0$. $Q(A) = 0$.
2. In B: target = $10 + 0 = 10$. Error = $10 - 0 = 10$. $Q(B) = 0 + 0.5(10) = 5$.

After second episode (with updated $Q(B) = 5$):

1. In A: target = $0 + 0.9(5) = 4.5$. Error = $4.5 - 0 = 4.5$. $Q(A) = 0 + 0.5(4.5) = 2.25$.
2. In B: target = $10 + 0 = 10$. Error = $10 - 5 = 5$. $Q(B) = 5 + 0.5(5) = 7.5$.

Answer: After 2 episodes: $Q(A) = 2.25$, $Q(B) = 7.5$

True values: $Q^*(B) = 10$, $Q^*(A) = 0.9 \times 10 = 9$. Converging toward these.

5.4 Q-Learning: Off-Policy TD Control

Q-Learning updates Q-values using the **maximum** Q-value of the next state, regardless of which action was actually taken:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)]$$

This is **off-policy** because the target uses $\max_{a'} Q(S_{t+1}, a')$ (the optimal action), while the behavior policy may explore randomly. Q-Learning learns the optimal policy directly, even while exploring.

Worked Example 37: Q-Learning Update

Given: Same Q-table as Example 34. $\alpha = 0.5$, $\gamma = 1.0$. Agent in state A takes action Right, gets $R = 5$, moves to B. In B, the actual action taken was Left, but Q-Learning uses the max.

Calculate: Updated $Q(A, \text{Right})$.

Solution:

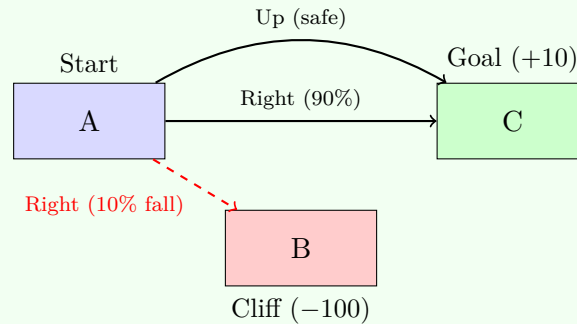
1. Current $Q(A, \text{Right}) = 1.0$
2. Q-Learning target: $R + \gamma \max_{a'} Q(B, a') = 5 + 1.0 \times 3.0 = 8.0$ (max is Right = 3.0)
3. TD error: $8.0 - 1.0 = 7.0$
4. Update: $Q(A, \text{Right}) = 1.0 + 0.5(7.0) = 4.5$

Answer: $Q(A, \text{Right}) = 4.5$

Key Difference from SARSA: Q-Learning target (8.0) > SARSA target (6.5 or 7.0). Q-Learning is more **optimistic** because it assumes the best next action. It learns the optimal policy even while exploring suboptimally.

Worked Example 38: Q-Learning vs SARSA Comparison

Given: A cliff-walking gridworld. States: A (start), B (cliff, reward -100), C (goal, reward +10). From A, action Right goes to C normally, but with 10% chance falls to B.



Q-values: $Q(A, \text{Right}) = 5$, $Q(A, \text{Up}) = 4$. $\epsilon = 0.1$.

Question: Which algorithm (Q-Learning or SARSA) learns to take action Up to avoid the cliff?

Solution:

1. **Q-Learning:** Updates using max next Q. Since the optimal path (Right without falling) has high value, Q-Learning learns $Q(A, \text{Right}) = 0.9(10) = 9$. But the behavior policy with $\epsilon = 0.1$ occasionally falls off the cliff.
2. **SARSA:** Updates using the actual next action. Since 10% of the time the agent falls (due to environment stochasticity or exploration), SARSA accounts for this risk. It learns a safer path (Up) with lower variance, even if the expected return is slightly lower.

Answer: SARSA learns the safer policy (Up); Q-Learning learns the optimal policy (Right) but suffers from occasional falls during exploration.

Interpretation: Q-Learning is more **risk-seeking** (optimistic about future actions); SARSA is more **risk-averse** (considers actual behavior including exploration).

Worked Example 39: Q-Learning Convergence

Given: A simple 3-state chain: $S \rightarrow A \rightarrow G$ (goal). Only one action per state. $R(S \rightarrow A) = 0$, $R(A \rightarrow G) = 10$. $\gamma = 0.9$, $\alpha = 0.1$. Initial $Q = 0$ everywhere.

After 100 episodes: What are the converged Q-values?

Solution:

1. $Q(G) = 0$ (terminal state)
2. $Q(A) = R + \gamma \max Q(G) = 10 + 0 = 10$ (converged)
3. $Q(S) = R + \gamma \max Q(A) = 0 + 0.9(10) = 9$ (converged)

Answer: $Q(S) = 9.0$, $Q(A) = 10.0$, $Q(G) = 0$

Interpretation: Q-Learning converges to the optimal values by propagating rewards backward from the goal. The discount factor $\gamma = 0.9$ reduces the value of earlier states proportionally.

6 Module 6: Function Approximation and Deep Reinforcement Learning

6.1 Drawbacks of Tabular Methods

Tabular methods (Q-tables, value tables) store an entry for every possible state (or state-action pair). This works well for small problems but suffers from major limitations:

- **Large State Spaces:** A modest 10x10 grid with 4 actions has $100 \times 4 = 400$ entries. A video game with 84×84 grayscale pixels at 256 levels has 256^{7056} states — completely intractable.
- **No Generalization:** In tabular methods, each state is treated independently. Changing the value of one state does not affect any other state, even if the states are very similar. This means the agent must visit every state many times to learn good values.

Function approximation solves both problems: instead of storing a table, we learn a **parameterized function** that maps states (or state-action pairs) to value estimates. This function can generalize across similar states and handle enormous state spaces.

6.2 Value Function Approximation

We approximate the value function v_π (or action-value q_π) using a parameterized function:

$$\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$$

where $\mathbf{w}$ is a vector of **weights** (parameters) learned through experience.

The goal is to minimize the **Mean Squared Value Error (MSVE)**:

$$\text{MSVE}(\mathbf{w}) = \sum_{s \in \mathcal{S}} \mu(s) [v_\pi(s) - \hat{v}(s, \mathbf{w})]^2$$

where $\mu(s)$ is a weighting factor (often the fraction of time spent in state s under policy π).

6.3 Gradient Descent Methods

To optimize the weights, we use **gradient descent** (or ascent for maximization):

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{1}{2} \alpha \nabla_{\mathbf{w}} [v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t)]^2$$

Using stochastic gradient descent (SGD), we update weights after each sample:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

Worked Example 40: Linear Parameterization Update

Given: A value function approximated as $\hat{v}(s, \mathbf{w}) = w_1 x_1(s) + w_2 x_2(s)$, where $\mathbf{x}(s) = [x_1, x_2]$ is the feature vector. Current weights: $\mathbf{w} = [2.0, -1.0]$. State s has features $[3, 2]$. True return $v_\pi(s) = 5$. $\alpha = 0.1$.

Calculate: Updated weights.

Solution:

1. Current estimate: $\hat{v}(s, \mathbf{w}) = 2.0(3) + (-1.0)(2) = 6 - 2 = 4$
2. Prediction error: $v_\pi(s) - \hat{v} = 5 - 4 = 1$
3. Gradients: $\nabla_w \hat{v} = [x_1, x_2] = [3, 2]$
4. Update: $\Delta \mathbf{w} = \alpha \cdot \text{error} \cdot \nabla_w \hat{v} = 0.1(1)[3, 2] = [0.3, 0.2]$
5. New weights: $\mathbf{w}_{\text{new}} = [2.0, -1.0] + [0.3, 0.2] = [2.3, -0.8]$

Verification: New estimate $= 2.3(3) + (-0.8)(2) = 6.9 - 1.6 = 5.3$, closer to 5.0.

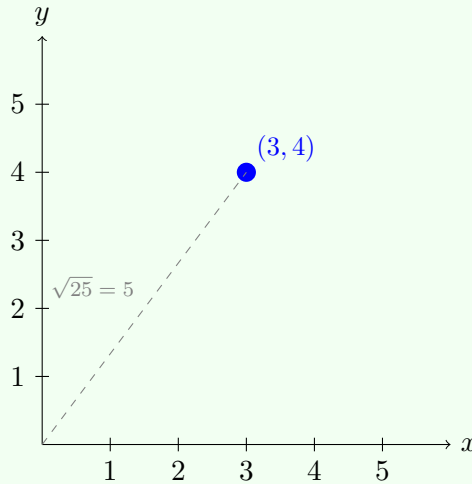
Answer: $\mathbf{w}_{\text{new}} = [2.3, -0.8]$

Interpretation: The weight update moves the estimated value toward the target. Features with larger values ($x_1 = 3$) receive larger updates because they contribute more to the prediction.

Worked Example 41: Feature Design for Function Approximation

Given: A gridworld agent at position (x, y) . We use linear FA with 3 features:

- $f_1 = x$ (horizontal position)
- $f_2 = y$ (vertical position)
- $f_3 = \sqrt{x^2 + y^2}$ (distance from origin)



Weights: $w_1 = 1.0$, $w_2 = -0.5$, $w_3 = 2.0$. Compute $\hat{v}$ for state $(3, 4)$.

Solution:

1. $f_1 = 3$, $f_2 = 4$, $f_3 = \sqrt{9 + 16} = \sqrt{25} = 5$
2. $\hat{v} = 1.0(3) + (-0.5)(4) + 2.0(5) = 3 - 2 + 10 = 11$

Answer: $\hat{v}(3, 4) = 11$

Interpretation: The value is a weighted sum of geometric features. This linear approximation generalizes to any (x, y) position without requiring a separate table entry. However, the quality depends heavily on good feature design — if the true value function is not well-approximated by these features, learning will be poor.

Worked Example 42: Neural Network Value Estimation

Given: A neural network approximates $Q(s, a)$ with a single hidden layer. Input: 4-dimensional state vector. Hidden: 3 neurons with ReLU. Output: Q-value. Current weights produce $Q(s, a_1) = 5.2$, $Q(s, a_2) = 7.8$. Target (from TD): $R + \gamma \max Q(s', a') = 10$.

Learning rate $\alpha = 0.01$. Backpropagation computes $\frac{\partial Q}{\partial w}$ for each weight.

Question: How does the weight update differ from linear FA?

Solution:

1. For the chosen action a_2 : error = $10 - 7.8 = 2.2$
2. Weight update: $\Delta w = \alpha \cdot \text{error} \cdot \frac{\partial Q}{\partial w} = 0.01(2.2)\nabla_w Q$
3. Unlike linear FA where $\nabla_w \hat{v} = \mathbf{x}(s)$, neural networks have **nonlinear** gradients through ReLU and hidden layers.
4. The gradient affects **all** weights in the network, not just the output layer weights for the active features.

Interpretation: Neural networks can learn **arbitrary** feature representations automatically, unlike linear FA where features must be hand-designed. This is the core advantage of Deep RL.

6.4 Deep Q-Network (DQN)

DQN combines Q-Learning with deep neural networks to approximate the action-value function $Q(s, a; \theta)$, where θ represents the neural network weights.

Key innovations of DQN:

1. **Experience Replay:** Stores past transitions (s, a, r, s') in a buffer and samples random mini-batches for training. This breaks correlation between consecutive samples and increases data efficiency.
2. **Target Network:** Uses a separate network (with frozen weights, updated periodically) to compute TD targets. This stabilizes learning by preventing the moving target problem.

The DQN loss function (Bellman update):

$$L(\theta) = \mathbb{E}[(R + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2]$$

where θ^- are the frozen target network weights.

6.5 Double DQN

Standard DQN tends to **overestimate** Q-values because the same network selects and evaluates actions. **Double DQN** decouples these roles:

1. Use the **online network** $Q(s', a'; \theta)$ to **select** the best action.
2. Use the **target network** $Q(s', a'; \theta^-)$ to **evaluate** that action.

The Double DQN update target:

$$Y_t^{\text{DoubleDQN}} = R_{t+1} + \gamma Q(S_{t+1}, \arg \max_a Q(S_{t+1}, a; \theta_t); \theta_t^-)$$

This reduces overestimation bias and often leads to more stable learning and better policies.

6.6 Actor-Critic Architecture

The **Actor-Critic** combines the benefits of policy-based and value-based methods:

- **Actor:** A policy network $\pi(a|s; \theta)$ that selects actions.
- **Critic:** A value network $V(s; \mathbf{w})$ (or $Q(s, a; \mathbf{w})$) that evaluates the actions taken.

The critic provides a **baseline** that reduces the variance of policy gradient updates. The advantage function:

$$A(s, a) = Q(s, a) - V(s)$$

tells how much better action a is compared to the average action in state s .

The actor updates using:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(a|s; \theta) \cdot A(s, a)$$

6.7 A2C and A3C

A2C (Advantage Actor-Critic): Synchronous version where multiple workers collect experience in parallel but updates are applied centrally and synchronously.

A3C (Asynchronous Advantage Actor-Critic): Multiple workers run in parallel on different CPU threads, each with its own copy of the environment. They compute gradients asynchronously and push them to a shared global network.

6.8 DDPG and PPO

DDPG (Deep Deterministic Policy Gradient) is designed for **continuous action spaces** (e.g., robotics, driving). It uses:

- A deterministic actor $\mu(s)$ that outputs a continuous action directly.
- A critic $Q(s, a)$ that evaluates state-action pairs.
- Experience replay and target networks (similar to DQN).

PPO (Proximal Policy Optimization) is the most widely used RL algorithm today. It improves upon vanilla policy gradients by restricting policy updates to prevent large, destabilizing changes:

Define the probability ratio:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

The PPO objective with clipping:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where ϵ is a hyperparameter (typically 0.1 or 0.2).

Worked Example 43: PPO Clipping

Given: Advantage $\hat{A}_t = +5$ (good action). Old policy probability: $\pi_{\text{old}} = 0.3$. New policy probability: $\pi_{\theta} = 0.5$. $\epsilon = 0.2$.

Calculate: The clipped objective value.

Solution:

1. Probability ratio: $r_t = \frac{0.5}{0.3} = 1.667$
2. Clip range: $[1 - 0.2, 1 + 0.2] = [0.8, 1.2]$
3. Since $r_t = 1.667 > 1.2$, the clipped ratio = 1.2
4. Unclipped term: $r_t \cdot \hat{A}_t = 1.667 \times 5 = 8.335$
5. Clipped term: $\text{clip}(r_t) \cdot \hat{A}_t = 1.2 \times 5 = 6.0$
6. Objective: $\min(8.335, 6.0) = 6.0$

Answer: Objective = 6.0 (clipped version used because ratio exceeded $1 + \epsilon$)

Interpretation: The new policy made action 67% more likely than the old policy ($r_t = 1.667$), but PPO clips at 1.2 (20% increase). The agent does not get extra reward for making the action *too* much more likely, preventing overly aggressive updates that could destabilize training.

Worked Example 44: PPO with Negative Advantage

Given: $\hat{A}_t = -4$ (bad action). $\pi_{\text{old}} = 0.4$. $\pi_{\theta} = 0.1$. $\epsilon = 0.2$.

Calculate: Clipped objective.

Solution:

1. Probability ratio: $r_t = \frac{0.1}{0.4} = 0.25$
2. Clip range: $[0.8, 1.2]$
3. Since $r_t = 0.25 < 0.8$, the clipped ratio = 0.8
4. Unclipped term: $r_t \cdot \hat{A}_t = 0.25 \times (-4) = -1.0$
5. Clipped term: $0.8 \times (-4) = -3.2$
6. Objective: $\min(-1.0, -3.2) = -3.2$

Answer: Objective = -3.2 (clipped version used because ratio fell below $1 - \epsilon$)

Interpretation: For a bad action, PPO wants to decrease its probability. The new policy already decreased it significantly ($r_t = 0.25$, probability dropped to 1/4). But PPO limits the penalty to $0.8 \times (-4) = -3.2$, preventing the policy from completely abandoning the action too quickly.

Worked Example 45: PPO Ratio Within Clip Range

Given: $\hat{A}_t = +3$. $\pi_{\text{old}} = 0.5$. $\pi_{\theta} = 0.55$. $\epsilon = 0.2$.

Calculate: Objective value.

Solution:

1. Probability ratio: $r_t = \frac{0.55}{0.5} = 1.1$
2. Clip range: $[0.8, 1.2]$
3. Since $0.8 < 1.1 < 1.2$, ratio is within clip range
4. Clipped ratio = $r_t = 1.1$ (no clipping)
5. Objective: $r_t \cdot \hat{A}_t = 1.1 \times 3 = 3.3$

Answer: Objective = 3.3 (unclipped, since ratio is within safe range)

Interpretation: When the policy change is moderate (10% increase, within $\pm 20\%$), PPO allows full gradient updates. This ensures the policy improves without dangerous overshooting.

6.9 Algorithm Comparison Summary

Algorithm	Type	Action Space	Learns	Key Trait
DQN	Value-based	Discrete	$Q(s, a)$	Replay buffer, target network
Double DQN	Value-based	Discrete	$Q(s, a)$	Reduces overestimation
REINFORCE	Policy-based	Both	$\pi(a s)$	Monte Carlo, high variance
A2C	Actor-Critic	Both	$\pi + V$	Synchronous parallel workers
A3C	Actor-Critic	Both	$\pi + V$	Asynchronous parallel workers
DDPG	Actor-Critic	Continuous	$\mu(s) + Q$	Replay buffer, deterministic
PPO	Actor-Critic	Both	$\pi + V$	Clipped updates, most popular

6.10 Prioritized Experience Replay

In standard experience replay, transitions are sampled uniformly from the replay buffer. **Prioritized Experience Replay** samples transitions with probability proportional to their **TD error** magnitude:

$$P(i) = \frac{|\delta_i|}{\sum_j |\delta_j|}$$

Transitions with large TD errors (surprising outcomes) are sampled more frequently, making learning more efficient. A small constant ϵ is added to prevent zero probabilities for low-error transitions.